

KEYS

1. Primary Key

A **Primary Key** uniquely identifies each record (row) in a table.

Features

- Cannot contain duplicate values
- Cannot contain NULL
- Only one primary key per table

Example

Customer Table

ID	Name	Aadhar_No	Phone_No	Email
1	Ravi	456712341234	9876543210	
2	Meena	789654123456	9123456780	

- **ID** is unique for every person.

So:

Primary Key = ID

2. Candidate Key

A **Candidate Key** is an attribute that can uniquely identify records.

A table can have **multiple candidate keys**.

(Minimal set of attributes (columns) that uniquely identifies each record in a table)

Example

Customer Table

ID	Name	Aadhar_No	Phone_No	Email
1	Ravi	456712341234	9876543210	
2	Meena	789654123456	9123456780	

In this table:

- ID → unique
- Aadhar_No → unique
- Phone_No → unique
- Email → unique

So all these are Candidate Keys.

Candidate Keys = ID, Aadhar_No, Phone_No, Email

3. Super Key

A **Super Key** is a set of one or more attributes that uniquely identify a record.

Example

Customer Table

ID	Name	Aadhar_No	Phone_No	Email
1	Ravi	456712341234	9876543210	
2	Meena	789654123456	9123456780	

Every Candidate Key is a Super Key, but every Super Key is not a Candidate Key

- ID
- Aadhar_No
- Phone_No
- Email
- ID + Name
- Email + Name

All these can identify a person uniquely.

So they are Super Keys.

4. Alternate Key

Candidate keys that are **not selected as Primary Key** are called Alternate Keys.

(Candidate key but not a primary key)

Example

Customer Table

ID	Name	Aadhar_No	Phone_No	Email
1	Ravi	456712341234	9876543210	
2	Meena	789654123456	9123456780	

If:

Primary Key = ID

Then:

Alternate Keys = Aadhar_No, Phone_No, Email

5. Composite Key

A key formed using **two or more attributes**.

Key composed of two or more columns that uniquely identifies single row in a table.

Example

Customer Table

ID	Name	Aadhar_No	Phone_No	Email
1	Ravi	456712341234	9876543210	
2	Meena	789654123456	9123456780	

- Name + Phone_No
(or)
- Name + Email

Together they can uniquely identify a record.

So these are Composite Keys.

6. Foreign Key

A **Foreign Key** creates a relationship between two tables.

It refers to the Primary Key of another table.

Example

Customer Table

ID	Name	Aadhar_No	Phone_No	Email
1	Ravi	456712341234	9876543210	
2	Meena	789654123456	9123456780	

Orders Table

Order_ID	Customer_ID	Product
O1	1	Laptop
O2	2	Mobile

Here:

- Customer_ID refers to ID in Customer table.

So:

Foreign Key = Customer_ID

It creates relationship between:

- Customer table
 - Orders table
-

7. Unique Key

A **Unique Key** ensures all values are unique.

Features

- Duplicate values are not allowed
- Usually allows one NULL value (DB dependent)

Example

Customer Table

ID	Name	Aadhar_No	Phone_No	Email
1	Ravi	456712341234	9876543210	
2	Meena	789654123456	9123456780	

- Aadhar_No
- Phone_No
- Email

These must be different for every person.

So they can be Unique Keys.

CONSTRAINTS in DBMS

Definition

Constraints are rules applied on table columns to maintain:

- data accuracy
- data consistency
- data integrity

They restrict invalid data from entering the database.

Types of Constraints in DBMS

1. NOT NULL
2. UNIQUE
3. PRIMARY KEY
4. FOREIGN KEY
5. CHECK
6. DEFAULT

EXPLICIT CONSTRAINTS

Definition

Explicit constraints are constraints that are **defined using the CONSTRAINT keyword with a custom name**.

Syntax

CONSTRAINT constraint_name constraint_type(column_name)

Example

```
CREATE TABLE TB(  
    eid INT,  
    ename VARCHAR(50),  
    CONSTRAINT pk PRIMARY KEY(eid)  
);
```

Here

Part	Meaning
CONSTRAINT	Keyword to define constraint explicitly
pk	User-defined constraint name
PRIMARY KEY(eid)	Constraint type

Use of Explicit Constraints

- ✓ Gives a custom name to constraints
- ✓ Easier to identify errors
- ✓ Easier to modify/drop constraints later
- ✓ Improves readability and maintenance

Example Advantage

Explicit Constraint

CONSTRAINT pk PRIMARY KEY(eid)

Error message:

ORA-00001: unique constraint (PK) violated

You can easily identify the constraint.

Without Explicit Constraint (Implicit)

eid INT PRIMARY KEY

Oracle automatically generates a random constraint name like:

SYS_C001234

which is harder to understand.

Short Note

☞ Explicit constraints are user-named constraints created using the CONSTRAINT keyword for better control, readability, and maintenance.

1. NOT NULL Constraint

Definition

Ensures a column cannot contain NULL values.

Syntax

column_name datatype NOT NULL

Example

```
CREATE TABLE Students(  
    ID INT,  
    Name VARCHAR2(50) NOT NULL  
);
```

Here:

- Name cannot be empty.

2. UNIQUE Constraint

Definition

Ensures all values in a column are unique.
Duplicates are not allowed.

Syntax

column_name datatype UNIQUE

Example

```
CREATE TABLE Students(  
    Email VARCHAR2(100) UNIQUE  
);
```

Two students cannot have same email.

3. PRIMARY KEY Constraint

Definition

Uniquely identifies each row in a table. (Primary Key = Unique + Not Null)

Features

- Unique
- Cannot be NULL

Syntax

column_name datatype PRIMARY KEY

Example

```
CREATE TABLE Students(  
    ID INT PRIMARY KEY,  
    Name VARCHAR2(50)  
);
```

ID uniquely identifies every student.

4. FOREIGN KEY Constraint

Definition

Creates relationship between two tables.
Foreign key refers to primary key of another table.

Syntax

```
FOREIGN KEY(column_name)  
REFERENCES parent_table(primary_key_column)
```

Example

```
CREATE TABLE Department(  
    Dept_ID INT PRIMARY KEY,  
    Dept_Name VARCHAR2(50)  
);
```

```
CREATE TABLE Students(  
    ID INT PRIMARY KEY,  
    Name VARCHAR2(50),  
    Dept_ID INT,  
    FOREIGN KEY(Dept_ID)  
    REFERENCES Department(Dept_ID)  
);
```

Dept_ID connects both tables.

5. CHECK Constraint

Definition

Restricts values based on a condition. (Displays data only if the condition is true)

Syntax

column_name datatype CHECK(condition)

Example

```
CREATE TABLE Students(  
    Age INT CHECK(Age >= 18)  
);
```

Only age 18 or above allowed.

6. DEFAULT Constraint

Definition

Assigns default value automatically if no value is provided.

Syntax

column_name datatype DEFAULT value

Example

```
CREATE TABLE Students(  
    City VARCHAR2(30) DEFAULT 'Coimbatore'  
);
```

If city is not given:

- Coimbatore is inserted automatically.

Full Example

```
CREATE TABLE Students(  
    ID INT PRIMARY KEY,  
    Name VARCHAR2(50) NOT NULL,  
    Email VARCHAR2(100) UNIQUE,  
    Age INT CHECK(Age >= 18),  
    City VARCHAR2(30) DEFAULT 'Coimbatore'  
);
```

Constraint	Purpose
NOT NULL	Prevents NULL values
UNIQUE	Prevents duplicate values
PRIMARY KEY	Unique + Not NULL
FOREIGN KEY	Creates table relationship
CHECK	Restricts condition
DEFAULT	Gives default value

DATA TYPES

Definition

Data types specify:

- what kind of data can be stored in a column
- size and format of data

They help maintain:

- data accuracy
- memory efficiency
- data validation

Types of Data Types in DBMS

1. Numeric Data Types
2. Character/String Data Types
3. Date & Time Data Types
4. Large Object Data Types
5. Boolean Data Type

1. Numeric Data Types

Used to store numbers.

Data Type	Description	Example
INT / INTEGER	Stores whole numbers	10
SMALLINT	Small integer values	100
BIGINT	Large integer values // not supported by Oracle but mysql supports	9876543210
DECIMAL(total no.of digits,no. of digits after decimal)	Fixed decimal numbers	86.56
FLOAT	Floating-point numbers	12.345
NUMBER(total no.of digits,no. of digits after decimal)	Stores any numeric value of size <= total no. of digits defined at creation of table	NUMBER(5,2)

Example

```
CREATE TABLE Numeric_Examples(  
    Age INT,  
    Marks INTEGER,  
    Small_Value SMALLINT,  
    Population NUMBER(10),  
    CGPA DECIMAL(4,2),  
    Percentage FLOAT,  
    Salary NUMBER(8,2)  
);
```

```
INSERT INTO Numeric_Examples  
VALUES(  
    21,  
    450,  
    120,  
    9876543210,  
    8.75,  
    92.56,  
    45000.75  
);
```

```
SELECT * FROM Numeric_Examples;
```

AGE	MARKS	SMALL_VALUE	POPULATION	CGPA	PERCENTAGE	SALARY
21	450	120	9876543210	8.75	92.56	45000.75

2. Character/String Data Types

Used to store text values.

Data Type	Description	Example
CHAR(n)	Fixed-length string	'A'
VARCHAR(n)	Variable-length string	'John'
VARCHAR2(n)	Oracle string datatype	'Emma'
TEXT	Large text & it's not supported by Oracle (use CLOB) but mysql supports	Paragraph

Example

```
CREATE TABLE Employee_Info(  
    Gender CHAR(1),  
    Name VARCHAR(50),  
    City VARCHAR2(50),  
    Description CLOB
```

```
);

INSERT INTO Employee_Info
VALUES(
    'M',
    'John',
    'Emma',
    'Senior developer working in full stack web development and cloud technologies.'
);

SELECT * FROM Employee_Info;
```

GEN	NAME	CITY	DESCRIPTION
M	John	Emma	Senior developer wor king in full stack w eb development and c loud technologies.

Difference: CHAR vs VARCHAR

CHAR

Fixed size

Wastes extra space

Faster sometimes

VARCHAR

Variable size

Saves memory

More flexible

3. Date & Time Data Types

Used to store dates and times.

Data Type	Description
DATE	Stores date
TIME	Stores time & it's not supported by Oracle (use CLOB) but mysql supports
DATETIME	Stores date & time & it's not supported by Oracle (use CLOB) but mysql supports
TIMESTAMP	Date + time + fractions

Example

```
CREATE TABLE Event_Details(
    Event_Name VARCHAR2(50),
    Event_Date DATE, // Stores only the date of event
    Start_Time DATE, // Stores date + time (used like time in Oracle)
    Event_DateTime TIMESTAMP, // Stores date + time + milliseconds
    Created_At TIMESTAMP // Stores record creation date and exact time automatically/manual
);
```

```

INSERT INTO Event_Details
VALUES(
    'Tech Meetup',
    TO_DATE('2026-05-22 10:30:00','YYYY-MM-DD HH24:MI:SS'),
    // Converts text → DATE (event date & time stored in DATE format)
    TO_DATE('2026-05-22 10:30:00','YYYY-MM-DD HH24:MI:SS'),
    // Stores same value again (used for Start_Time)
    TO_TIMESTAMP('2026-05-22 10:30:00.123','YYYY-MM-DD HH24:MI:SS.FF3'),
    // Converts text → TIMESTAMP (includes milliseconds)
    // FF3 → Format Mask For Fractional Seconds (Store 3 digits of milliseconds )
    CURRENT_TIMESTAMP
    // Automatically stores current system date & time
);

SELECT * FROM Event_Details;

```

EVENT_NAME	EVENT_DATE	START_TIME	EVENT_DATETIME	CREATED_AT
Tech Meetup	22-MAY-26	22-MAY-26	22-MAY-26 10.30.00.123000 AM	22-MAY-26 02.01.00.001000 PM

4. Large Object Data Types

Used to store large data.

Data Type	Stores
BLOB (Binary Large Object)	Images, videos, files
CLOB (Character Large Object)	Large text
NCLOB (National Character Large Object)	Unicode text

Example

```

CREATE TABLE Student_Prof (
    student_id NUMBER PRIMARY KEY,
    student_name VARCHAR2(50),
    photo BLOB,
    bio CLOB,
    feedback NCLOB
);

INSERT INTO Student_Prof
VALUES (
    1,
    'Arun',
    EMPTY_BLOB(),
    'Arun is a bright student with strong programming skills.',
    N'அருண் மிகவும் திறமையான மாணவர்' && 'Excellent performance'
);

```

```
DESC Student_Prof;
select * from Student_Prof;
```

Table created.

1 row created.

Name	Null?	Type
STUDENT_ID	NOT NULL	NUMBER
STUDENT_NAME		VARCHAR2(50)
PHOTO		BLOB
BIO		CLOB
FEEDBACK		NCLOB

SP2-0678: Column or attribute type can not be displayed by SQL*Plus

```
SQL> select * from Student_Prof;

STUDENT_ID STUDENT_NAME
-----
1 Arun

Arjun is a bright student with strong programming skills.
????? ??????? ????????? ?????? ?? Excellent performance
```

5. Boolean Data Type

Stores:

- TRUE
- FALSE

Some DBMS support it directly.

Oracle does NOT have a native BOOLEAN datatype in SQL tables (it exists only in PL/SQL, not in normal table columns).

So in Oracle, we simulate BOOLEAN using:

- NUMBER(1) → 1 = TRUE, 0 = FALSE
- or CHAR(1) → 'Y' / 'N'

Example

Is_Active BOOLEAN

Common Oracle Data Types

Oracle Datatype	Purpose
NUMBER	Numeric values
VARCHAR2	Strings
CHAR	Fixed text
DATE	Date values

Oracle Datatype	Purpose
TIMESTAMP	Date and time
CLOB	Large text
BLOB	Binary files
Full Example CREATE TABLE Employee(ID NUMBER(5), Name VARCHAR2(50), Salary NUMBER(8,2), DOB DATE, Resume CLOB);	
Quick Summary	
Category	Examples
Numeric	INT, FLOAT, DECIMAL
String	CHAR, VARCHAR
Date/Time	DATE, TIMESTAMP
Large Objects	BLOB, CLOB
Boolean	TRUE/FALSE

DATA MODELS

DDL (Data Definition Language)

DDL commands are used to **define, modify, and remove database structures** like tables, views, indexes, etc.

List of DDL Commands

Command	Purpose
CREATE	Creates database objects
ALTER	Modifies existing objects
DROP	Deletes objects permanently
TRUNCATE	Removes all rows from a table
RENAME	Renames database objects
COMMENT	Adds comments/descriptions (mainly Oracle)

Common Example Table

Student Table

REG_NO	NAME	AGE	COURSE	DATE_OF_BIRTH
101	Arun	19	CSE	2006-05-10
102	Meena	20	IT	2005-08-15
103	Kavin	21	ECE	2004-11-20
104	Divya	22	AI&DS	2003-03-25
105	Hari	18	MECH	2007-01-12

1. CREATE Command

Definition

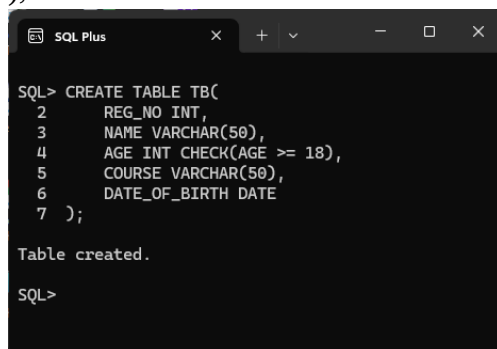
Used to create a new table, database, view, etc.

Oracle Syntax

```
CREATE TABLE table_name (  
    column1 datatype constraints,  
    column2 datatype constraints  
);
```

Oracle Example

```
CREATE TABLE TB(  
    REG_NO INT,  
    NAME VARCHAR(50),  
    AGE INT CHECK(AGE >= 18),  
    COURSE VARCHAR(50),  
    DATE_OF_BIRTH DATE  
);
```



```
SQL Plus  
SQL> CREATE TABLE TB(  
2     REG_NO INT,  
3     NAME VARCHAR(50),  
4     AGE INT CHECK(AGE >= 18),  
5     COURSE VARCHAR(50),  
6     DATE_OF_BIRTH DATE  
7 );  
  
Table created.  
  
SQL>
```

MySQL Syntax

```
CREATE TABLE table_name (  
    column1 datatype constraints,  
    column2 datatype constraints  
);
```

MySQL Example

```
CREATE TABLE TB(  
    REG_NO INT,  
    NAME VARCHAR(50),  
    AGE INT CHECK(AGE >= 18),
```

```
COURSE VARCHAR(50),  
DATE_OF_BIRTH DATE  
);
```

CREATE DATABASE

Definition

Creates a new database.

MySQL Syntax

```
CREATE DATABASE database_name;
```

MySQL Example

```
CREATE DATABASE CollegeDB;
```

Use Database

```
USE CollegeDB;
```

Oracle Equivalent

Oracle does not usually use CREATE DATABASE in normal SQL practice.
Instead:

- Database is created by DBA (Database Administrator)
- Users/schema are commonly created

Example:

```
CREATE USER TB_STUDENT IDENTIFIED BY pass123;  
GRANT CONNECT, RESOURCE TO TB_STUDENT;
```

CREATE VIEW

Definition

A VIEW is a virtual table created using a query from one or more tables.
It does not store actual data permanently.
It shows data taken from the original table(s).

Type	Meaning
-------------	----------------

Simple View	Based on one table
-------------	--------------------

Complex View	Based on multiple tables/functions
--------------	------------------------------------

Uses of VIEW (Short)

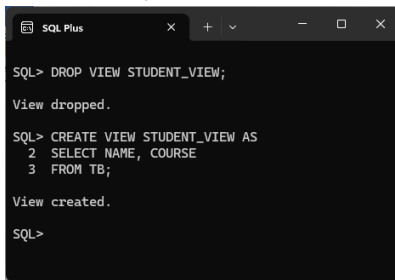
- Hides sensitive data
- Simplifies complex queries
- Shows only required data
- Improves security
- Reuses frequently used queries

Oracle Syntax

```
CREATE VIEW view_name AS  
SELECT columns  
FROM table_name  
WHERE condition;
```

Oracle Example

```
CREATE VIEW STUDENT_VIEW AS  
SELECT NAME, COURSE  
FROM TB;
```



```
SQL> DROP VIEW STUDENT_VIEW;  
View dropped.  
SQL> CREATE VIEW STUDENT_VIEW AS  
2  SELECT NAME, COURSE  
3  FROM TB;  
View created.  
SQL>
```

MySQL Syntax

```
CREATE VIEW view_name AS  
SELECT columns  
FROM table_name  
WHERE condition;
```

MySQL Example

```
CREATE VIEW STUDENT_VIEW AS  
SELECT NAME, COURSE  
FROM TB;
```

How to See Data from View

```
SELECT * FROM STUDENT_VIEW;
```

View Output

NAME	COURSE
Arun	CSE
Meena	IT
Kavin	ECE
Divya	AI&DS
Hari	MECH

2. ALTER Command

Definition

The ALTER command is used to modify the structure of an existing table.

Using ALTER, we can:

- Add columns
- Modify columns
- Drop columns
- Rename columns

1. ADD COLUMN

Definition

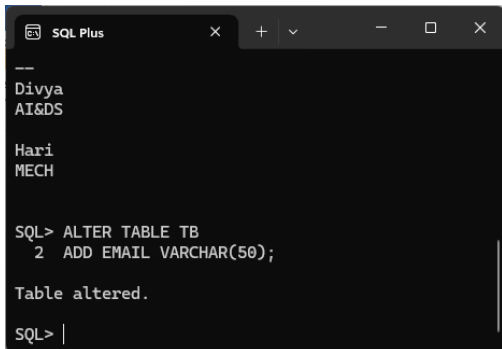
Adds a new column to an existing table.

Oracle Syntax

```
ALTER TABLE table_name  
ADD column_name datatype;
```

Oracle Example

```
ALTER TABLE TB  
ADD EMAIL VARCHAR2(50);
```

A screenshot of a SQL Plus window titled 'SQL Plus'. The window shows a list of records with columns 'NAME' and 'COURSE'. The records are: Divya (AI&DS), Hari (MECH), and a blank row. Below the records, the SQL command 'ALTER TABLE TB ADD EMAIL VARCHAR(50);' is entered. The output 'Table altered.' is displayed, followed by the SQL prompt 'SQL> |'.

MySQL Syntax

```
ALTER TABLE table_name  
ADD column_name datatype;
```

MySQL Example

```
ALTER TABLE TB  
ADD EMAIL VARCHAR(50);
```

Result

REG_NO	NAME	AGE	COURSE	DATE_OF_BIRTH	EMAIL
--------	------	-----	--------	---------------	-------

2. MODIFY COLUMN

Definition

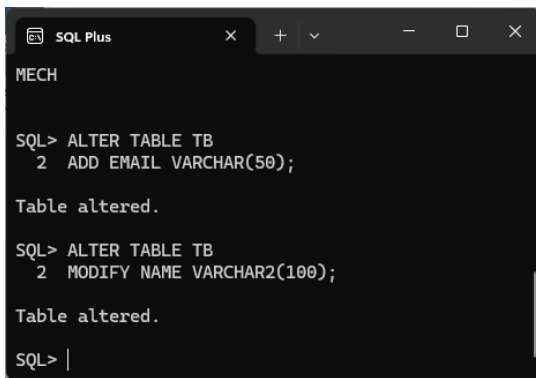
Changes datatype or size of an existing column.

Oracle Syntax

```
ALTER TABLE table_name  
MODIFY column_name new_datatype;
```

Oracle Example

```
ALTER TABLE TB  
MODIFY NAME VARCHAR2(100);
```

A screenshot of a SQL Plus terminal window. The window has a title bar with 'SQL Plus' and standard window controls. The terminal shows a prompt 'MECH' followed by two SQL commands. The first command is 'SQL> ALTER TABLE TB' followed by a subprompt '2' and 'ADD EMAIL VARCHAR(50);'. The response is 'Table altered.'. The second command is 'SQL> ALTER TABLE TB' followed by a subprompt '2' and 'MODIFY NAME VARCHAR2(100);'. The response is 'Table altered.'. The prompt 'SQL>' is visible at the bottom.

```
MECH

SQL> ALTER TABLE TB
  2  ADD EMAIL VARCHAR(50);

Table altered.

SQL> ALTER TABLE TB
  2  MODIFY NAME VARCHAR2(100);

Table altered.

SQL> |
```

MySQL Syntax

ALTER TABLE table_name
MODIFY column_name new_datatype;

MySQL Example

ALTER TABLE TB
MODIFY NAME VARCHAR(100);

Meaning

Changes size of NAME column from 50 to 100.

3. DROP COLUMN

Definition

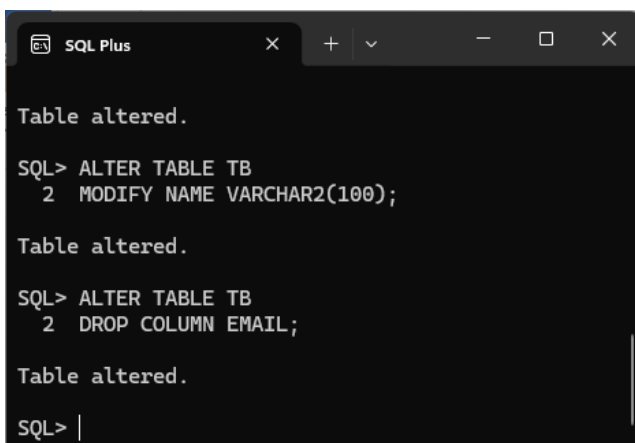
Deletes a column from the table.

Oracle Syntax

ALTER TABLE table_name
DROP COLUMN column_name;

Oracle Example

ALTER TABLE TB
DROP COLUMN EMAIL;

A screenshot of a SQL Plus terminal window. The window has a title bar with 'SQL Plus' and standard window controls. The terminal shows a prompt 'MECH' followed by two SQL commands. The first command is 'SQL> ALTER TABLE TB' followed by a subprompt '2' and 'MODIFY NAME VARCHAR2(100);'. The response is 'Table altered.'. The second command is 'SQL> ALTER TABLE TB' followed by a subprompt '2' and 'DROP COLUMN EMAIL;'. The response is 'Table altered.'. The prompt 'SQL>' is visible at the bottom.

```
MECH

SQL> ALTER TABLE TB
  2  MODIFY NAME VARCHAR2(100);

Table altered.

SQL> ALTER TABLE TB
  2  DROP COLUMN EMAIL;

Table altered.

SQL> |
```

MySQL Syntax

ALTER TABLE table_name
DROP COLUMN column_name;

MySQL Example

```
ALTER TABLE TB  
DROP COLUMN EMAIL;
```

4. RENAME COLUMN

Definition

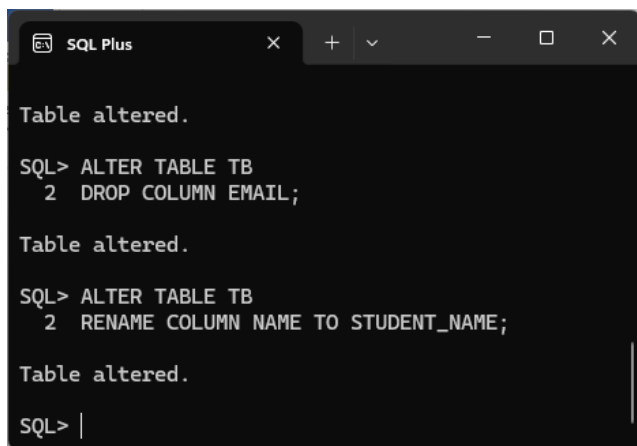
Changes the name of an existing column.

Oracle Syntax

```
ALTER TABLE table_name  
RENAME COLUMN old_column_name TO new_column_name;
```

Oracle Example

```
ALTER TABLE TB  
RENAME COLUMN NAME TO STUDENT_NAME;
```



```
SQL Plus  
Table altered.  
SQL> ALTER TABLE TB  
2 DROP COLUMN EMAIL;  
Table altered.  
SQL> ALTER TABLE TB  
2 RENAME COLUMN NAME TO STUDENT_NAME;  
Table altered.  
SQL> |
```

MySQL Syntax

```
ALTER TABLE table_name  
RENAME COLUMN old_column_name TO new_column_name;
```

MySQL Example

```
ALTER TABLE TB  
RENAME COLUMN NAME TO STUDENT_NAME;
```

Result

REG_NO	STUDENT_NAME	AGE	COURSE	DATE_OF_BIRTH
--------	--------------	-----	--------	---------------

Short Note

ALTER is used to change the structure of an existing table without deleting the table.

5. Adding Constraint to a Column

Definition

Constraints can be added to an existing column using ALTER TABLE.

Syntax

```
ALTER TABLE table_name  
ADD CONSTRAINT constraint_name  
constraint_type(column_name);
```

Example — Add PRIMARY KEY Constraint

```
ALTER TABLE TB  
ADD CONSTRAINT pk  
PRIMARY KEY(eid);
```

6. Drop Constraint from a Column

Syntax (Oracle / MySQL)

```
ALTER TABLE table_name  
DROP CONSTRAINT constraint_name;
```

3. DROP Command

Definition

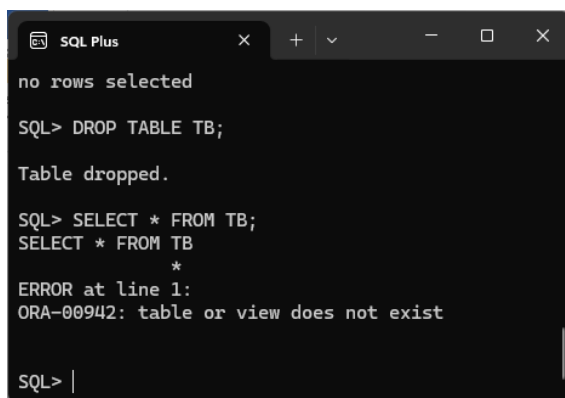
Deletes the entire table structure and data permanently.

Oracle Syntax

```
DROP TABLE table_name;
```

Oracle Example

```
DROP TABLE TB;
```



```
SQL Plus  
no rows selected  
  
SQL> DROP TABLE TB;  
  
Table dropped.  
  
SQL> SELECT * FROM TB;  
SELECT * FROM TB  
      *  
ERROR at line 1:  
ORA-00942: table or view does not exist  
  
SQL> |
```

MySQL Syntax

```
DROP TABLE table_name;
```

MySQL Example

```
DROP TABLE TB;
```

4. TRUNCATE Command

Definition

Deletes all rows from a table but keeps the table structure.

Oracle Syntax

```
TRUNCATE TABLE table_name;
```

Oracle Example

```
TRUNCATE TABLE TB;
```

```
SQL Plus
SQL> ALTER TABLE TB
  2  RENAME COLUMN NAME TO STUDENT_NAME;

Table altered.

SQL> TRUNCATE TABLE TB;

Table truncated.

SQL> SELECT * FROM TB;

no rows selected

SQL> |
```

MySQL Syntax

TRUNCATE TABLE table_name;

MySQL Example

TRUNCATE TABLE TB;

5. RENAME Command

RENAME Command

Definition

The RENAME command is used to change the name of:

- tables
- columns
- database objects

1. RENAME TABLE

Oracle Syntax

RENAME old_table_name TO new_table_name;

Oracle Example

RENAME TB TO STUDENT_TB;

```
SQL Plus
SQL>
SQL> INSERT INTO TB
  2  VALUES (105, 'Hari', 18, 'MECH',
  3  TO_DATE('2007-01-12', 'YYYY-MM-DD'));

1 row created.

SQL> RENAME TB TO STUDENT_TB;

Table renamed.

SQL> |
```

MySQL Syntax

RENAME TABLE old_table_name TO new_table_name;

MySQL Example

RENAME TABLE TB TO STUDENT_TB;

2. RENAME COLUMN

Oracle Syntax

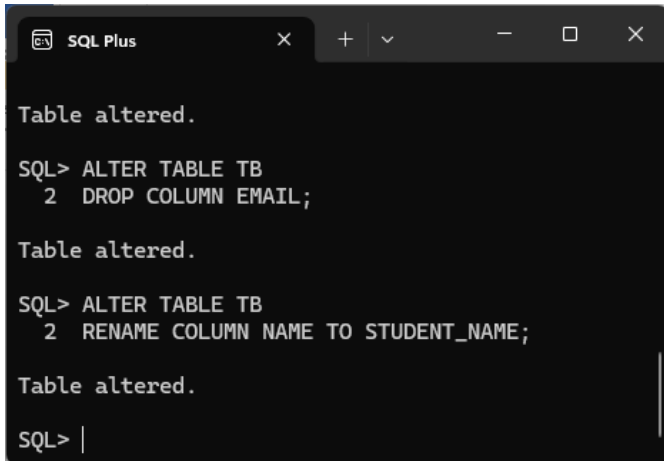
ALTER TABLE table_name

RENAME COLUMN old_column_name TO new_column_name;

Oracle Example

ALTER TABLE TB

RENAME COLUMN NAME TO STUDENT_NAME;

A screenshot of a SQL Plus terminal window. The window title is "SQL Plus". The terminal shows the following sequence of commands and responses: 1. "Table altered." 2. "SQL> ALTER TABLE TB" followed by "2 DROP COLUMN EMAIL;" on the next line. 3. "Table altered." 4. "SQL> ALTER TABLE TB" followed by "2 RENAME COLUMN NAME TO STUDENT_NAME;" on the next line. 5. "Table altered." 6. "SQL> |" with a cursor at the end of the line.

```
SQL Plus
Table altered.
SQL> ALTER TABLE TB
2 DROP COLUMN EMAIL;
Table altered.
SQL> ALTER TABLE TB
2 RENAME COLUMN NAME TO STUDENT_NAME;
Table altered.
SQL> |
```

MySQL Syntax

ALTER TABLE table_name

RENAME COLUMN old_column_name TO new_column_name;

MySQL Example

ALTER TABLE TB

RENAME COLUMN NAME TO STUDENT_NAME;

Short Note

RENAME is used to change the name of a table or column without affecting the data.

6. COMMENT Command

Definition

The COMMENT command is used to add descriptions/comments to:

- tables
- columns

It helps users understand the purpose of database objects.

(Mainly used in Oracle)

Oracle Syntax

COMMENT ON TABLE table_name IS 'comment';

Oracle Example

COMMENT ON TABLE STUDENT_TB IS 'Stores student details';

```
SQL Plus
COMMENT ON TABLE SUDENT_TB IS 'Stores student details'
*
ERROR at line 1:
ORA-00942: table or view does not exist

SQL> COMMENT ON TABLE STUDENT_TB IS 'Stores student details';

Comment created.

SQL> |
```

MySQL Alternative

MySQL usually adds comments during table creation.

```
CREATE TABLE Student (
    student_id INT PRIMARY KEY COMMENT 'Student ID',
    name VARCHAR(50)
);
```

2. COMMENT ON COLUMN

Oracle Syntax

COMMENT ON COLUMN table_name.column_name IS 'comment';

Oracle Example

COMMENT ON COLUMN STUDENT_TB.STUDENT_NAME IS 'Stores student name';

```
SQL Plus

SQL> COMMENT ON TABLE STUDENT_TB IS 'Stores student details';

Comment created.

SQL> COMMENT ON COLUMN STUDENT_TB.STUDENT_NAME IS 'Stores student name';

Comment created.

SQL> |
```

MySQL Comment Syntax

In MySQL, comments are usually added while creating the table.

MySQL Syntax

```
CREATE TABLE table_name(
    column_name datatype COMMENT 'comment'
);
```

MySQL Example

```
CREATE TABLE TB(
    REG_NO INT COMMENT 'Student Register Number',
    NAME VARCHAR(50) COMMENT 'Student Name',
    AGE INT COMMENT 'Student Age',
    COURSE VARCHAR(50) COMMENT 'Course Name'
```

);

DML (Data Manipulation Language)

Definition

DML commands are used to **manipulate data inside a table** (insert, modify, delete, retrieve rows). They work on **records (rows)**, not table structure.

LIST OF DML COMMANDS

Command	Definition
INSERT	Adds new records into table
UPDATE	Modifies existing records
DELETE	Removes records from table

1. INSERT COMMAND

Definition

Used to add new rows into a table.

Syntax

Oracle

```
INSERT INTO table_name VALUES (value1, value2, ...);
```

MySQL

```
INSERT INTO table_name VALUES (value1, value2, ...);
```

Example

```
INSERT INTO STUDENT_TB  
VALUES (101, 'Arun', 19, 'CSE',  
TO_DATE('2006-05-10','YYYY-MM-DD'));
```

```
INSERT INTO STUDENT_TB  
VALUES (102, 'Meena', 20, 'IT',  
TO_DATE('2005-08-15','YYYY-MM-DD'));
```

```
INSERT INTO STUDENT_TB  
VALUES (103, 'Kavin', 21, 'ECE',  
TO_DATE('2004-11-20','YYYY-MM-DD'));
```

```
INSERT INTO STUDENT_TB  
VALUES (104, 'Divya', 22, 'AI&DS',  
TO_DATE('2003-03-25','YYYY-MM-DD'));
```

```
INSERT INTO STUDENT_TB  
VALUES (105, 'Hari', 18, 'MECH',  
TO_DATE('2007-01-12','YYYY-MM-DD'));
```

```
SQL Plus
3  TO_DATE('2006-05-10','YYYY-MM-DD'));
1 row created.

SQL>
SQL> INSERT INTO STUDENT_TB
2  VALUES (102, 'Meena', 20, 'IT',
3  TO_DATE('2005-08-15','YYYY-MM-DD'));

1 row created.

SQL>
SQL> INSERT INTO STUDENT_TB
2  VALUES (103, 'Kavin', 21, 'ECE',
3  TO_DATE('2004-11-20','YYYY-MM-DD'));

1 row created.

SQL>
SQL> INSERT INTO STUDENT_TB
2  VALUES (104, 'Divya', 22, 'AI&DS',
3  TO_DATE('2003-03-25','YYYY-MM-DD'));

1 row created.

SQL>
SQL> INSERT INTO STUDENT_TB
2  VALUES (105, 'Hari', 18, 'MECH',
3  TO_DATE('2007-01-12','YYYY-MM-DD'));

1 row created.
```

To get input from user

```
INSERT INTO table_name VALUES (&col_name_1, '&col_name_2', ...);
// single quotes are used only in case of char values
```

Example

```
INSERT INTO STUDENT_TB
VALUES (
    &REG_NO,
    '&STUDENT_NAME',
    &AGE,
    '&COURSE',
    TO_DATE('&DATE_OF_BIRTH','YYYY-MM-DD')
);
```

```
SQL Plus
SQL> INSERT INTO STUDENT_TB
2 VALUES (
3   &REG_NO,
4   '&STUDENT_NAME',
5   &AGE,
6   '&COURSE',
7   TO_DATE('&DATE_OF_BIRTH','YYYY-MM-DD')
8 );
Enter value for reg_no: 001
old 3:   &REG_NO,
new 3:   001,
Enter value for student_name: Latchaya_V
old 4:   '&STUDENT_NAME',
new 4:   'Latchaya_V',
Enter value for age: 19
old 5:   &AGE,
new 5:   19,
Enter value for course: FULL STACK
old 6:   '&COURSE',
new 6:   'FULL STACK',
Enter value for date_of_birth: 2007-05-15
old 7:   TO_DATE('&DATE_OF_BIRTH','YYYY-MM-DD')
new 7:   TO_DATE('2007-05-15','YYYY-MM-DD')

1 row created.

SQL> select * from STUDENT_TB where REG_NO = 001;

  REG_NO STUDENT_NAME                                AGE
-----
COURSE                                DATE_OF_B
1 Latchaya_V                                19
FULL STACK                                15-MAY-07
```

Copy Values from One Table to Another Table

☞ INSERT INTO ... SELECT is used to copy data from one table to another table.

☞ Data types should be same & the varchar should have enough memory space to copy the data

Syntax

```
INSERT INTO table2
SELECT * FROM table1;
```

Example

Suppose:

Table 1

STUDENT_TB

Table 2

STUDENT_BACKUP

Copy All Records

```
INSERT INTO STUDENT_BACKUP
SELECT * FROM STUDENT_TB;
```

Meaning

☞ Copies all rows from STUDENT_TB into STUDENT_BACKUP.

Copy Specific Columns

Syntax

```
INSERT INTO table2(column1, column2)
SELECT column1, column2
FROM table1;
```

Example

```
INSERT INTO STUDENT_BACKUP(REG_NO, STUDENT_NAME)
SELECT REG_NO, STUDENT_NAME
FROM STUDENT_TB;
```

Copy Specific Rows

Example

```
INSERT INTO STUDENT_BACKUP
SELECT *
FROM STUDENT_TB
WHERE COURSE = 'CSE';
```

2. UPDATE COMMAND

Definition

Used to modify existing data in a table.

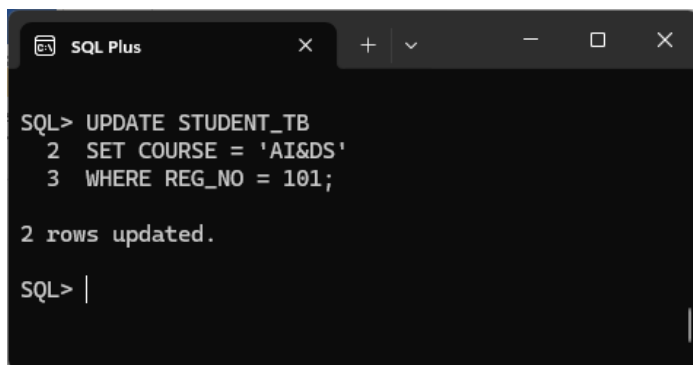
Syntax

Oracle / MySQL

```
UPDATE table_name
SET column_name = value
WHERE condition;
```

Example

```
UPDATE STUDENT_TB
SET COURSE = 'AI&DS'
WHERE REG_NO = 101;
```

A screenshot of a SQL Plus terminal window. The window title is "SQL Plus". The terminal shows the following text: SQL> UPDATE STUDENT_TB, followed by a line number 2 and the text SET COURSE = 'AI&DS', followed by a line number 3 and the text WHERE REG_NO = 101;. Below this, it says "2 rows updated." and then "SQL> |" with a cursor.

3. DELETE COMMAND

Definition

Used to remove records from a table.

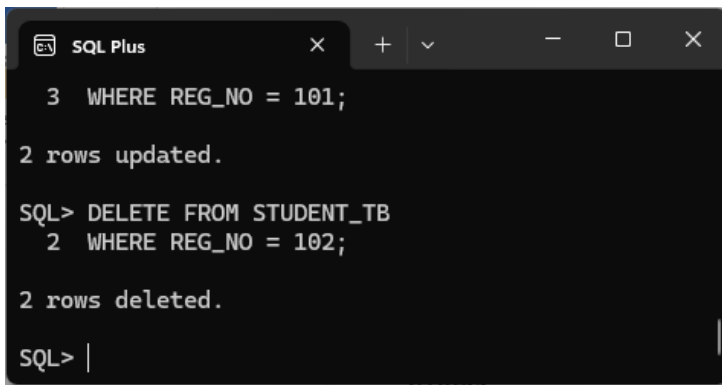
Syntax

Oracle / MySQL

```
DELETE FROM table_name
WHERE condition;
```

Example

```
DELETE FROM STUDENT_TB
WHERE REG_NO = 102;
```



```
SQL Plus
3 WHERE REG_NO = 101;
2 rows updated.
SQL> DELETE FROM STUDENT_TB
2 WHERE REG_NO = 102;
2 rows deleted.
SQL> |
```

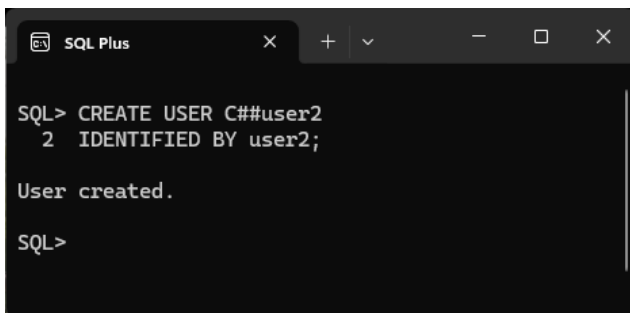
CREATE USER

Syntax

CREATE USER C##User_name
IDENTIFIED BY password;

Example

CREATE USER C##user2
IDENTIFIED BY user2;



```
SQL Plus
SQL> CREATE USER C##user2
2 IDENTIFIED BY user2;
User created.
SQL>
```

DROP USER

Definition

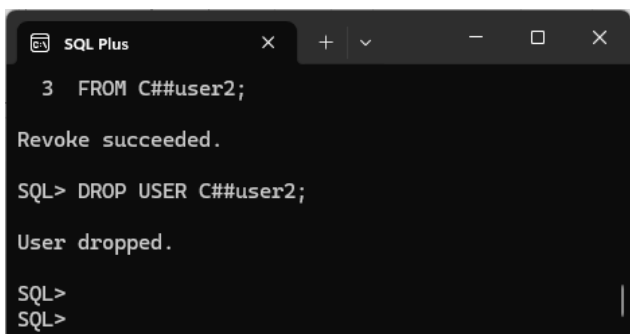
Used to remove/delete a user from Oracle.

Syntax

DROP USER user_name;

Example

DROP USER C##user2;



```
SQL Plus
3 FROM C##user2;
Revoke succeeded.
SQL> DROP USER C##user2;
User dropped.
SQL>
SQL>
```

If User Has Tables/Objects

Use:

Syntax

```
DROP USER C##User_name CASCADE;
```

Example

```
DROP USER C##user2 CASCADE;
```

Meaning of CASCADE

CASCADE deletes:

- tables
- views
- procedures
- all objects owned by the user

along with the user account.

Short Note

☞ DROP USER removes a user from Oracle database.

☞ CASCADE removes user and all associated database objects.

DCL (Data Control Language)

Definition

DCL commands are used to **control access and permissions** in the database.

☞ DCL commands are used to control access to database objects using GRANT and REVOKE.

☞ It manages **who can access data and what actions they can perform.**

- DCL controls **security and access**
- Used mostly by DBA (Database Administrator)
- Works with users and privileges

Main DCL Commands

Command	Purpose
GRANT	Gives permissions to users
REVOKE	Removes permissions from users

Real-Life Analogy

Command	Meaning
GRANT	Giving key to a room
REVOKE	Taking back the key

1. GRANT COMMAND

Definition

Used to give permissions to a user.

To give access to user to perform one or more operations

Syntax (Oracle / MySQL)

GRANT privileges

ON table_name

TO user_name;

Example

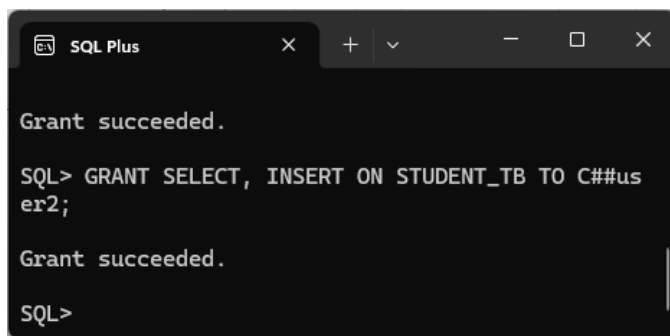
GRANT SELECT, INSERT

ON STUDENT_TB

TO C##user2;

☞ Meaning:

- student user can **read and insert data** into TB



```
SQL Plus
Grant succeeded.
SQL> GRANT SELECT, INSERT ON STUDENT_TB TO C##user2;
Grant succeeded.
SQL>
```

To give access to user to perform all operations

Syntax (Oracle / MySQL)

GRANT ALL PRIVILEGES

ON table_name

TO user_name;

Example

GRANT ALL PRIVILEGES

ON STUDENT_TB

TO C##user2;

(OR)

GRANT ALL

ON STUDENT_TB

TO C##user2;



```
SQL> GRANT ALL PRIVILEGES ON STUDENT_TB TO C##user2;
Grant succeeded.
```

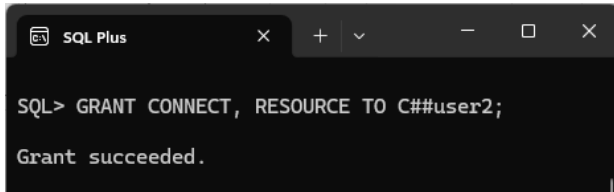
Grant Connect / Resource

Syntax

```
GRANT privilege1, privilege2  
FROM user_name;
```

Example

```
GRANT CONNECT, RESOURCE TO C##user2;
```



```
SQL Plus  
SQL> GRANT CONNECT, RESOURCE TO C##user2;  
Grant succeeded.
```

Privilege	Meaning
CONNECT	Allows login
RESOURCE	Allows creating tables, views, etc.
DBA	Full database admin control
CREATE SESSION	Allows login/session
CREATE TABLE	Allows creating tables
CREATE VIEW	Allows creating views
CREATE PROCEDURE	Allows creating procedures
CREATE SEQUENCE	Allows creating sequences
CREATE TRIGGER	Allows creating triggers

2. REVOKE COMMAND

Definition

Used to remove permissions from a user.

To revoke access from user for one or more operations

Syntax (Oracle / MySQL)

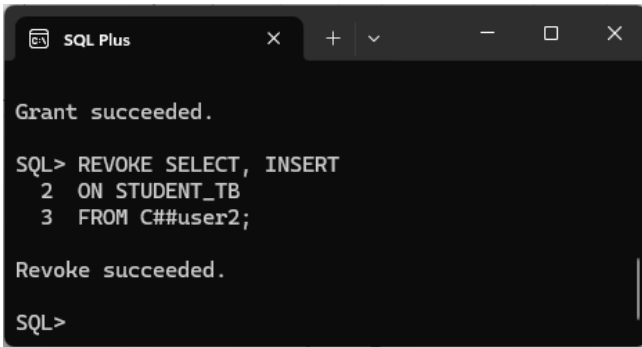
```
REVOKE privilege_name  
ON table_name  
FROM user_name;
```

Example

```
REVOKE SELECT, INSERT  
ON STUDENT_TB  
FROM C##user2;
```

☞ Meaning:

- removes select & insert permission from student



```
SQL Plus
Grant succeeded.
SQL> REVOKE SELECT, INSERT
2 ON STUDENT_TB
3 FROM C##user2;
Revoke succeeded.
SQL>
```

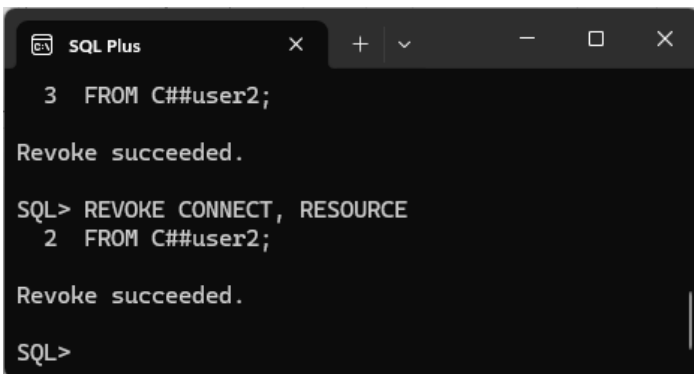
Revoke Connect / Resource

Syntax

```
REVOKE privilege1, privilege2
FROM user_name;
```

Example

```
REVOKE CONNECT, RESOURCE
FROM C##user2;
```



```
SQL Plus
3 FROM C##user2;
Revoke succeeded.
SQL> REVOKE CONNECT, RESOURCE
2 FROM C##user2;
Revoke succeeded.
SQL>
```

To revoke access from user for all operations

Syntax

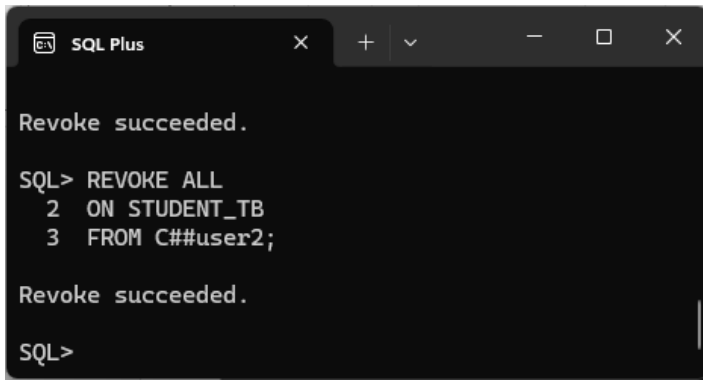
```
REVOKE ALL
ON table_name
FROM user_name;
```

Example

```
REVOKE ALL
ON STUDENT_TB
FROM C##user2;
```

(OR)

```
REVOKE ALL PRIVILEGES
ON STUDENT_TB
FROM C##user2;
```



```
SQL Plus
Revoke succeeded.

SQL> REVOKE ALL
2  ON STUDENT_TB
3  FROM C##user2;

Revoke succeeded.

SQL>
```

TCL (Transaction Control Language)

Definition

TCL commands are used to control transactions in the database.

☞ It manages transaction changes made using:

- INSERT
- UPDATE
- DELETE
- TCL controls transaction management
- Used to save or rollback changes
- Works mainly with DML operations

Main TCL Commands

Command	Purpose
COMMIT	Permanently saves changes
ROLLBACK	Undoes changes
SAVEPOINT	Creates a point for partial rollback

Real-Life Analogy

Command	Meaning
COMMIT	Saving a document permanently
ROLLBACK	Undoing recent changes
SAVEPOINT	Creating a checkpoint

1. COMMIT COMMAND

Definition

Used to permanently save changes in the database.

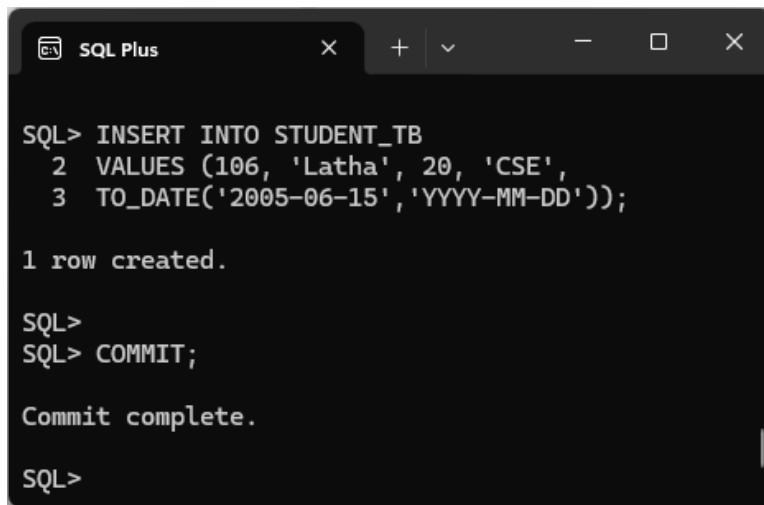
☞ Once committed, changes cannot be rolled back.

Syntax (Oracle / MySQL)

COMMIT;

Example

```
INSERT INTO STUDENT_TB  
VALUES (106, 'Latha', 20, 'CSE',  
TO_DATE('2005-06-15','YYYY-MM-DD'));  
COMMIT;
```



```
SQL> INSERT INTO STUDENT_TB  
  2  VALUES (106, 'Latha', 20, 'CSE',  
  3  TO_DATE('2005-06-15','YYYY-MM-DD'));  
  
1 row created.  
  
SQL>  
SQL> COMMIT;  
  
Commit complete.  
  
SQL>
```

☞ Meaning:

- record is permanently saved in database

2. ROLLBACK COMMAND

Definition

Used to undo changes that are not yet committed.

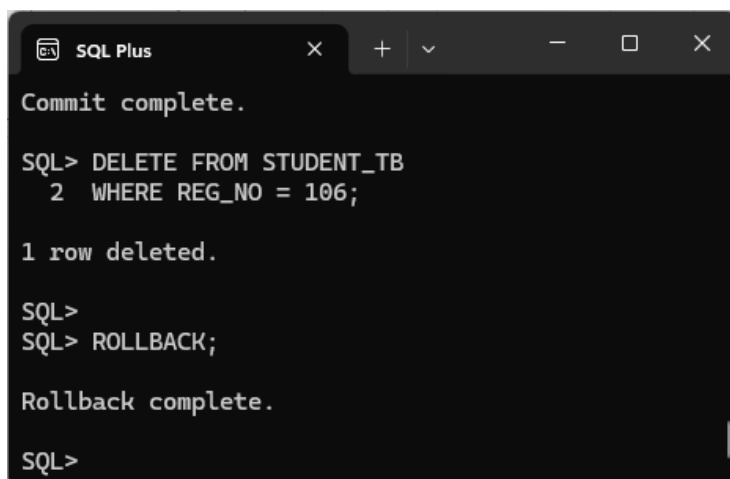
☞ Restores database to previous state.

Syntax (Oracle / MySQL)

```
ROLLBACK;
```

Example

```
DELETE FROM TB  
WHERE REG_NO = 106;  
ROLLBACK;
```



```
Commit complete.  
  
SQL> DELETE FROM STUDENT_TB  
  2  WHERE REG_NO = 106;  
  
1 row deleted.  
  
SQL>  
SQL> ROLLBACK;  
  
Rollback complete.  
  
SQL>
```

☞ Meaning:

- deleted record is restored back

3. SAVEPOINT COMMAND

Definition

Used to create a temporary checkpoint inside a transaction.

☞ Helps rollback partially instead of undoing all changes.

Syntax (Oracle / MySQL)

SAVEPOINT savepoint_name;

Example

```
INSERT INTO STUDENT_TB
VALUES (107, 'Nila', 19, 'IT',
TO_DATE('2006-02-10','YYYY-MM-DD'));
```

SAVEPOINT A;

```
INSERT INTO STUDENT_TB
VALUES (108, 'Ravi', 21, 'ECE',
TO_DATE('2004-09-18','YYYY-MM-DD'));
```

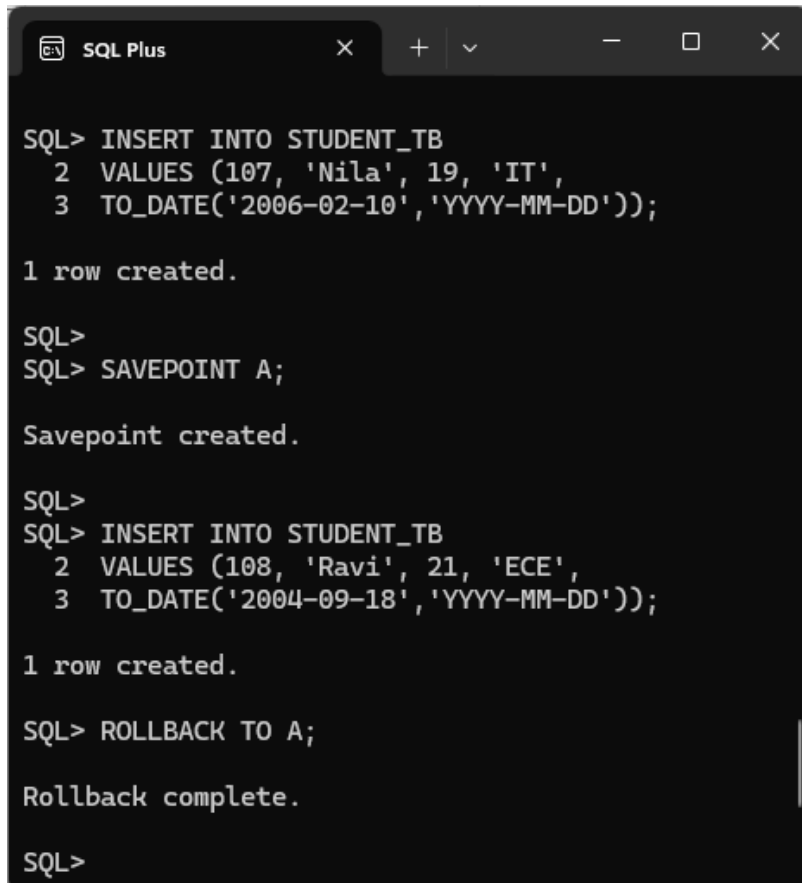
Rollback to SAVEPOINT

Syntax

ROLLBACK TO savepoint_name;

Example

ROLLBACK TO A;



```
SQL Plus
SQL> INSERT INTO STUDENT_TB
  2  VALUES (107, 'Nila', 19, 'IT',
  3  TO_DATE('2006-02-10','YYYY-MM-DD'));

1 row created.

SQL>
SQL> SAVEPOINT A;

Savepoint created.

SQL>
SQL> INSERT INTO STUDENT_TB
  2  VALUES (108, 'Ravi', 21, 'ECE',
  3  TO_DATE('2004-09-18','YYYY-MM-DD'));

1 row created.

SQL> ROLLBACK TO A;

Rollback complete.

SQL>
```

☞ Meaning:

- removes changes made after savepoint A
- keeps previous changes

Important Points

Command	Effect
COMMIT	Saves changes permanently
ROLLBACK	Undoes uncommitted changes
SAVEPOINT	Creates checkpoint

DQL (Data Query Language)

Definition

DQL commands are used to retrieve/query data from the database.

- DQL is used for data retrieval
- Works mainly with tables and views
- Used to display records from database

Main DQL Command

Command	Purpose
SELECT	Retrieves data from table

Real-Life Analogy

Command Meaning

SELECT Searching and viewing information from a file

SELECT COMMAND

Definition

Used to retrieve/display data from a table.

1. Select All Columns

Syntax

```
SELECT *  
FROM table_name;
```

Example

```
SELECT *  
FROM STUDENT_TB;
```

```
SQL Plus
SQL> SELECT *
2 FROM STUDENT_TB;

      REG_NO STUDENT_NAME
      AGE
-----
COURSE
DATE_OF_B
-----
      101 Arun
      19
CSE
10-MAY-06

      102 Meena
      20
IT
15-AUG-05

      103 Kavın
      21
ECE
20-NOV-04

      REG_NO STUDENT_NAME
      AGE
-----
COURSE
DATE_OF_B
```

```
SQL Plus
-----
      104 Divya
      22
AI&DS
25-MAR-03

      105 Hari
      18
MECH
12-JAN-07

      106 Latha
      20
CSE
15-JUN-05

      REG_NO STUDENT_NAME
      AGE
-----
COURSE
DATE_OF_B
-----
      107 Nila
      19
IT
10-FEB-06

7 rows selected.
```

Meaning

Displays all records and all columns from table.

2. Select Specific Columns

Syntax

```
SELECT column_name
FROM table_name;
```

Example

```
SELECT STUDENT_NAME, COURSE
FROM STUDENT_TB;
```

```
SQL Plus
SQL> SELECT STUDENT_NAME, COURSE
2 FROM STUDENT_TB;

STUDENT_NAME
-----
--
COURSE
-----
--
Arun
CSE

Meena
IT

Kavin
ECE

STUDENT_NAME
-----
--
COURSE
-----
--
Divya
AI&DS

Hari
MECH

Latha
CSE

STUDENT_NAME
-----
--
COURSE
-----
--
Nila
IT

7 rows selected.

SQL> |
```

Meaning

Displays only selected columns.

3. SELECT using WHERE Clause

Syntax

```
SELECT *
FROM table_name
WHERE condition;
```

Example

```
SELECT *
FROM STUDENT_TB
WHERE COURSE = 'CSE';
```

```

SQL Plus
SQL> SELECT *
2 FROM STUDENT_TB
3 WHERE COURSE = 'CSE';

  REG_NO STUDENT_NAME
  AGE
-----
COURSE
DATE_OF_B
-----
101 Arun
19
CSE
10-MAY-06
106 Latha
20
CSE
15-JUN-05

SQL>

```

Meaning

Displays records matching the condition.

4. SELECT using ORDER BY

Syntax

```

SELECT *
FROM table_name
ORDER BY column_name;

```

Example

```

SELECT *
FROM STUDENT_TB
ORDER BY AGE;

```

```

SQL Plus
SQL> SELECT *
2 FROM STUDENT_TB
3 ORDER BY AGE;

  REG_NO STUDENT_NAME
  AGE
-----
105 Hari
18
MECH
12-JAN-07
101 Arun
19
CSE
10-MAY-06
107 Nila
19
IT
10-FEB-06

  REG_NO STUDENT_NAME
  AGE
-----
COURSE
DATE_OF_B
-----

```

```

SQL Plus
-----
102 Meena
20
IT
15-AUG-05
106 Latha
20
CSE
15-JUN-05
103 Kavin
21
ECE
20-NOV-04

  REG_NO STUDENT_NAME
  AGE
-----
COURSE
DATE_OF_B
-----
104 Divya
22
AI&DS
25-MAR-03

7 rows selected.

SQL>

```

Meaning

Displays records sorted by AGE.

Important Points

Feature	Meaning
*	Select all columns
WHERE	Filters records
ORDER BY	Sorts records

CONCATENATION OPERATOR ...

|| → is known as concatenation operation

Used to merge two / more cols & display as a single one

Anything enclosed within single quote will display on the output

Example

select first_name || ' ' || last_name || ' Meow' from employees where DEPARTMENT_ID = 90;

FIRST_NAME " " LAST_NAME "MEOW"
Steven King Meow
Neena Kochhar Meow
Lex De Haan Meow

select first_name || ' ' || last_name || ' Meow' as NAME from employees where DEPARTMENT_ID = 90;

NAME
Steven King Meow
Neena Kochhar Meow
Lex De Haan Meow

DUMMY TABLE → DUAL

To perform arithmetic operations

Example

select (15*5 + 21*8) from dual;

(15*5+21*8)
243

OPERATORS IN SQL

Operators in SQL → Operators are used to specify conditions in SQL statements.

Example Table ...

```
CREATE TABLE STUDENT_TB(  
    REG_NO INT,  
    STUDENT_NAME VARCHAR(50),  
    AGE INT,  
    COURSE VARCHAR(50)  
);  
INSERT INTO STUDENT_TB  
VALUES (101, 'Arun', 19, 'CSE',  
TO_DATE('2006-05-10','YYYY-MM-DD'));  
  
INSERT INTO STUDENT_TB  
VALUES (102, 'Meena', 20, 'IT',  
TO_DATE('2005-08-15','YYYY-MM-DD'));  
  
INSERT INTO STUDENT_TB  
VALUES (103, 'Kavin', 21, 'ECE',  
TO_DATE('2004-11-20','YYYY-MM-DD'));
```

```
INSERT INTO STUDENT_TB  
VALUES (104, 'Divya', 22, 'AI&DS',  
TO_DATE('2003-03-25','YYYY-MM-DD'));  
  
INSERT INTO STUDENT_TB  
VALUES (105, 'Hari', 18, 'MECH',  
TO_DATE('2007-01-12','YYYY-MM-DD'));  
  
INSERT INTO STUDENT_TB  
VALUES (106, 'Latha', 20, 'CSE',  
TO_DATE('2005-06-15','YYYY-MM-DD'));  
  
INSERT INTO STUDENT_TB  
VALUES (128, 'Latchaya_V', 19, 'FULL  
STACK',  
TO_DATE('2007-05-15','YYYY-MM-DD'));
```

```
CREATE TABLE STUDENT_BACKUP(  
    REG_NO INT,  
    STUDENT_NAME VARCHAR2(50),  
    AGE INT,  
    COURSE VARCHAR2(50),  
    DATE_OF_BIRTH DATE  
);  
  
INSERT INTO STUDENT_BACKUP  
VALUES (201, 'Ravi', 21, 'CSE',  
TO_DATE('2004-02-14','YYYY-MM-DD'));  
  
INSERT INTO STUDENT_BACKUP  
VALUES (202, 'Nila', 20, 'IT',  
TO_DATE('2005-07-19','YYYY-MM-DD'));  
  
INSERT INTO STUDENT_BACKUP  
VALUES (203, 'Vignesh', 22, 'CYBER SECURITY',  
TO_DATE('2003-11-25','YYYY-MM-DD'));
```

1. > (GREATER THAN)

Definition

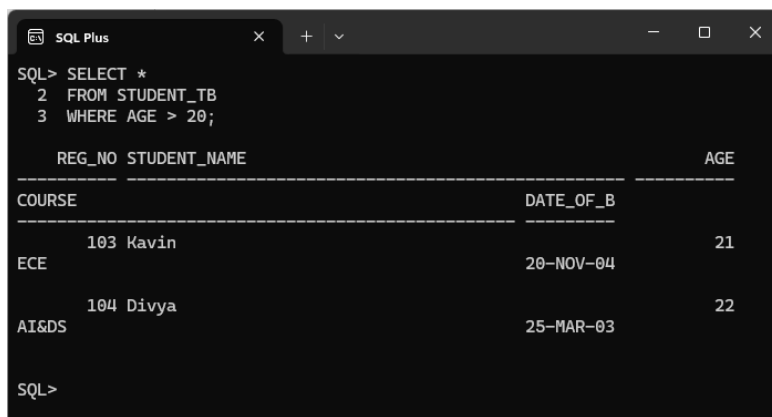
Checks whether a value is greater than another value.

Syntax

```
SELECT *  
FROM table_name  
WHERE column_name > value;
```

Example

```
SELECT *  
FROM STUDENT_TB  
WHERE AGE > 20;
```



The screenshot shows a SQL Plus window with the following content:

```
SQL> SELECT *  
  2  FROM STUDENT_TB  
  3  WHERE AGE > 20;
```

REG_NO	STUDENT_NAME	AGE
103	Kavin	21
104	Divya	22

The output also includes a header for 'COURSE' and 'DATE_OF_B' which are not explicitly shown in the table above but are part of the full output. The terminal shows the prompt 'SQL>' at the bottom.

2. < (LESS THAN)

Definition

Checks whether a value is less than another value.

Syntax

```
SELECT *  
FROM table_name  
WHERE column_name < value;
```

Example

```
SELECT *  
FROM STUDENT_TB  
WHERE AGE < 20;
```

```
SQL Plus
SQL> SELECT *
  2 FROM STUDENT_TB
  3 WHERE AGE < 20;

  REG_NO STUDENT_NAME                                AGE
-----
COURSE                                DATE_OF_B
-----
101 Arun                                19
CSE                                10-MAY-06
105 Hari                                18
MECH                                12-JAN-07
128 Latchaya_V                                19
FULL STACK                                15-MAY-07

SQL> |
```

3. >= (GREATER THAN OR EQUAL TO)

Definition

Checks whether value is greater than or equal to another value.

Syntax

```
SELECT *
FROM table_name
WHERE column_name >= value;
```

Example

```
SELECT *
FROM STUDENT_TB
WHERE AGE >= 21;
```

```
SQL Plus
SQL> SELECT *
  2 FROM STUDENT_TB
  3 WHERE AGE >= 21;

  REG_NO STUDENT_NAME                                AGE
-----
COURSE                                DATE_OF_B
-----
103 Kavin                                21
ECE                                20-NOV-04
104 Divya                                22
AI&DS                                25-MAR-03

SQL> |
```

4. <= (LESS THAN OR EQUAL TO)

Definition

Checks whether value is less than or equal to another value.

Syntax

```
SELECT *
FROM table_name
WHERE column_name <= value;
```

Example

```
SELECT *  
FROM STUDENT_TB  
WHERE AGE <= 20;
```



The screenshot shows a SQL Plus window with the following content:

```
SQL> SELECT *  
2 FROM STUDENT_TB  
3 WHERE AGE <= 20;
```

REG_NO	STUDENT_NAME	AGE
101	Arun	19
102	Meena	20
105	Hari	18
106	Latha	20
128	Latchaya_V	19

The results are displayed in two separate blocks. The first block shows three rows of data, and the second block shows two rows of data. The columns are REG_NO, STUDENT_NAME, and AGE.

5. <> (NOT EQUAL TO)

Definition

Checks whether two values are not equal.

Syntax

```
SELECT *  
FROM table_name  
WHERE column_name <> value;
```

Example

```
SELECT *  
FROM STUDENT_TB  
WHERE COURSE <> 'CSE';
```

```
SQL Plus
SQL> SELECT *
2 FROM STUDENT_TB
3 WHERE COURSE <> 'CSE';
```

REG_NO	STUDENT_NAME	AGE
102	Meena	20
103	Kavin	21
104	Divya	22
105	Hari	18
128	Latchaya_V	19

```
SQL> |
```

6. = (EQUAL TO)

Definition

Checks whether two values are equal.

Syntax

```
SELECT *
FROM table_name
WHERE column_name = value;
```

Example

```
SELECT *
FROM STUDENT_TB
WHERE COURSE = 'CSE';
```

```
SQL Plus
SQL> SELECT *
2 FROM STUDENT_TB
3 WHERE COURSE = 'CSE';
```

REG_NO	STUDENT_NAME	AGE
101	Arun	19
106	Latha	20

```
SQL>
```

7. BETWEEN

Definition

Checks whether value lies within a range.

Syntax

```
SELECT *  
FROM table_name  
WHERE column_name BETWEEN value1 AND value2;
```

Example

```
SELECT *  
FROM STUDENT_TB  
WHERE AGE BETWEEN 19 AND 21;
```



```
SQL> SELECT *  
2 FROM STUDENT_TB  
3 WHERE AGE BETWEEN 19 AND 21;
```

	REG_NO	STUDENT_NAME	AGE
COURSE			
	101	Arun	19
CSE			10-MAY-06
	102	Meena	20
IT			15-AUG-05
	103	Kavin	21
ECE			20-NOV-04

	REG_NO	STUDENT_NAME	AGE
COURSE			
	106	Latha	20
CSE			15-JUN-05
	128	Latchaya_V	19
FULL STACK			15-MAY-07

```
SQL>
```

8. NOT BETWEEN

Definition

Checks whether value lies outside a range.

Syntax

```
SELECT *  
FROM table_name  
WHERE column_name NOT BETWEEN value1 AND value2;
```

Example

```
SELECT *  
FROM STUDENT_TB  
WHERE AGE NOT BETWEEN 18 AND 20;
```

```
SQL Plus
SQL> SELECT *
2 FROM STUDENT_TB
3 WHERE AGE NOT BETWEEN 18 AND 20;

  REG_NO STUDENT_NAME                                AGE
-----
COURSE                                DATE_OF_B
-----
ECE      103 Kavin                                21
AI&DS    104 Divya                                22
          25-MAR-03
```

9. IN

Definition

Checks whether a value matches any value in a list.

Syntax

```
SELECT *
FROM table_name
WHERE column_name IN(value1, value2);
```

Example

```
SELECT *
FROM STUDENT_TB
WHERE COURSE IN('CSE', 'IT');
```

```
SQL Plus
SQL> SELECT *
2 FROM STUDENT_TB
3 WHERE COURSE IN('CSE', 'IT');

  REG_NO STUDENT_NAME                                AGE
-----
COURSE                                DATE_OF_B
-----
CSE      101 Arun                                19
          10-MAY-06
IT       102 Meena                                20
          15-AUG-05
CSE      106 Latha                                20
          15-JUN-05
```

10. NOT IN

Definition

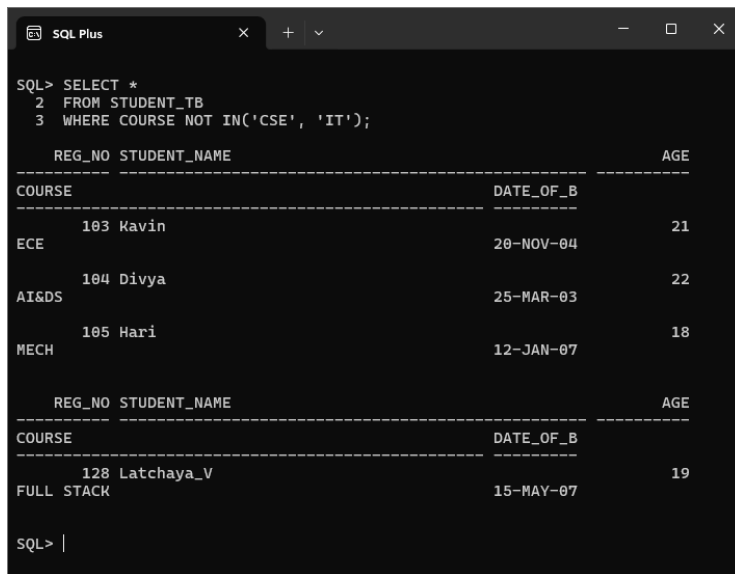
Checks whether value does not exist in a list.

Syntax

```
SELECT *
FROM table_name
WHERE column_name NOT IN(value1, value2);
```

Example

```
SELECT *  
FROM STUDENT_TB  
WHERE COURSE NOT IN('CSE', 'IT');
```



```
SQL> SELECT *  
  2  FROM STUDENT_TB  
  3  WHERE COURSE NOT IN('CSE', 'IT');
```

REG_NO	STUDENT_NAME	AGE
103	Kavin	21
104	Divya	22
105	Hari	18

REG_NO	STUDENT_NAME	AGE
128	Latchaya_V	19

```
SQL> |
```

11. IS NULL

Definition

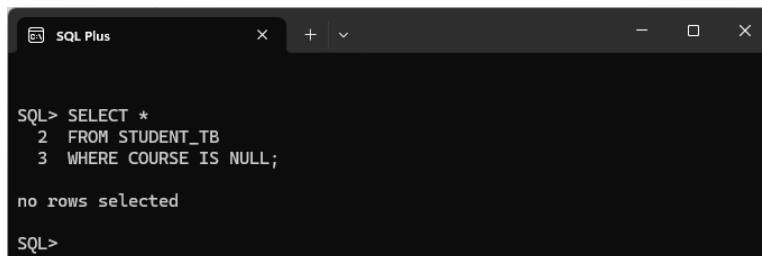
Checks for NULL (empty) values.

Syntax

```
SELECT *  
FROM table_name  
WHERE column_name IS NULL;
```

Example

```
SELECT *  
FROM STUDENT_TB  
WHERE COURSE IS NULL;
```



```
SQL> SELECT *  
  2  FROM STUDENT_TB  
  3  WHERE COURSE IS NULL;
```

no rows selected

```
SQL>
```

12. IS NOT NULL

Definition

Checks whether column contains non-null values.

Syntax

```
SELECT *  
FROM table_name  
WHERE column_name IS NOT NULL;
```

Example

```
SELECT *  
FROM STUDENT_TB  
WHERE COURSE IS NOT NULL;
```



```
SQL Plus  
SQL> SELECT *  
2 FROM STUDENT_TB  
3 WHERE COURSE IS NOT NULL;  
  
  REG_NO STUDENT_NAME                                AGE  
-----  
COURSE                                DATE_OF_B  
-----  
CSE      101 Arun                                     19  
IT        102 Meena                                    20  
ECE       103 Kavin                                    21  
  
  REG_NO STUDENT_NAME                                AGE  
-----  
COURSE                                DATE_OF_B  
-----  
AIGDS    104 Divya                                    22  
MECH      105 Hari                                     18  
CSE       106 Latha                                    20  
  
  REG_NO STUDENT_NAME                                AGE  
-----  
COURSE                                DATE_OF_B  
-----  
128 Latchaya_V                                     19  
FULL STACK  
15-MAY-07  
  
7 rows selected.  
SQL>
```

13. LIKE

Definition

Used for pattern matching.

Symbol	Meaning
--------	---------

%	Any number of characters
_	Single character

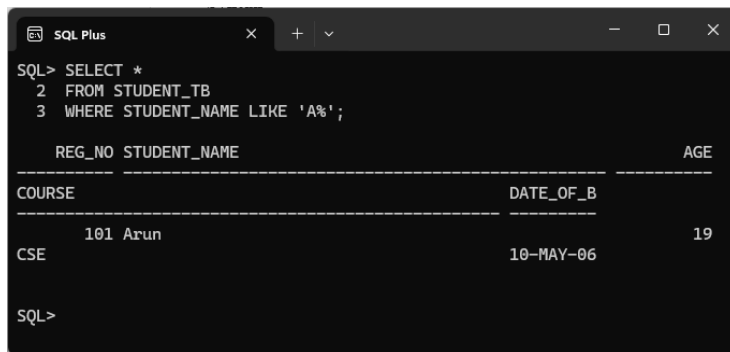
Syntax

```
SELECT *  
FROM table_name  
WHERE column_name LIKE pattern;
```

Example 1

```
SELECT *  
FROM STUDENT_TB  
WHERE STUDENT_NAME LIKE 'A%';
```

☞ Names starting with A

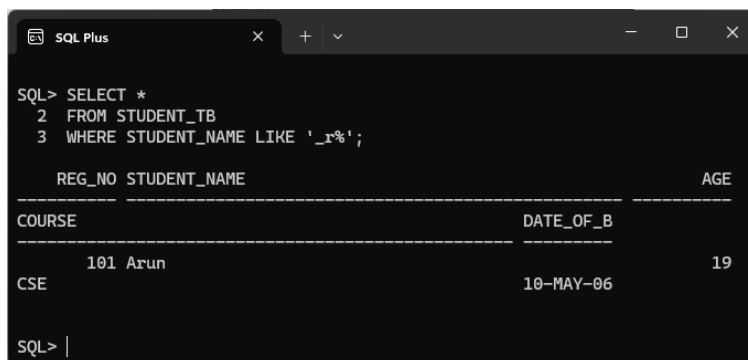


```
SQL Plus  
SQL> SELECT *  
2 FROM STUDENT_TB  
3 WHERE STUDENT_NAME LIKE 'A%';  
  
REG_NO STUDENT_NAME AGE  
-----  
COURSE DATE_OF_B  
-----  
101 Arun 19  
CSE 10-MAY-06  
  
SQL>
```

Example 2

```
SELECT *  
FROM STUDENT_TB  
WHERE STUDENT_NAME LIKE '_r%';
```

☞ Second letter should be r

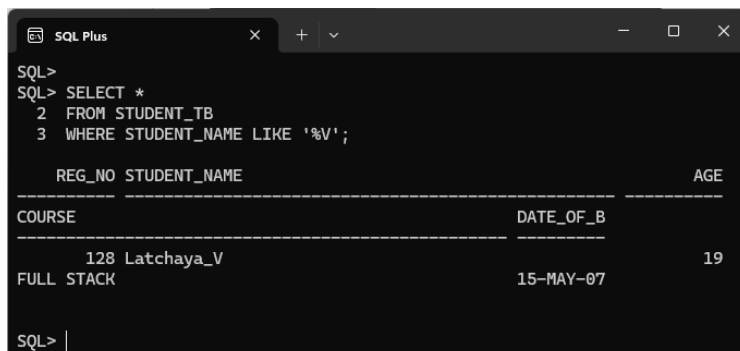


```
SQL Plus  
SQL> SELECT *  
2 FROM STUDENT_TB  
3 WHERE STUDENT_NAME LIKE '_r%';  
  
REG_NO STUDENT_NAME AGE  
-----  
COURSE DATE_OF_B  
-----  
101 Arun 19  
CSE 10-MAY-06  
  
SQL> |
```

Example 3

```
SELECT *  
FROM STUDENT_TB  
WHERE STUDENT_NAME LIKE '%V';
```

☞ Names ending with V



```
SQL Plus  
SQL>  
SQL> SELECT *  
2 FROM STUDENT_TB  
3 WHERE STUDENT_NAME LIKE '%V';  
  
REG_NO STUDENT_NAME AGE  
-----  
COURSE DATE_OF_B  
-----  
128 Latchaya_V 19  
FULL STACK 15-MAY-07  
  
SQL> |
```

14. EXISTS

Definition

EXISTS checks only whether the subquery returns rows or not.

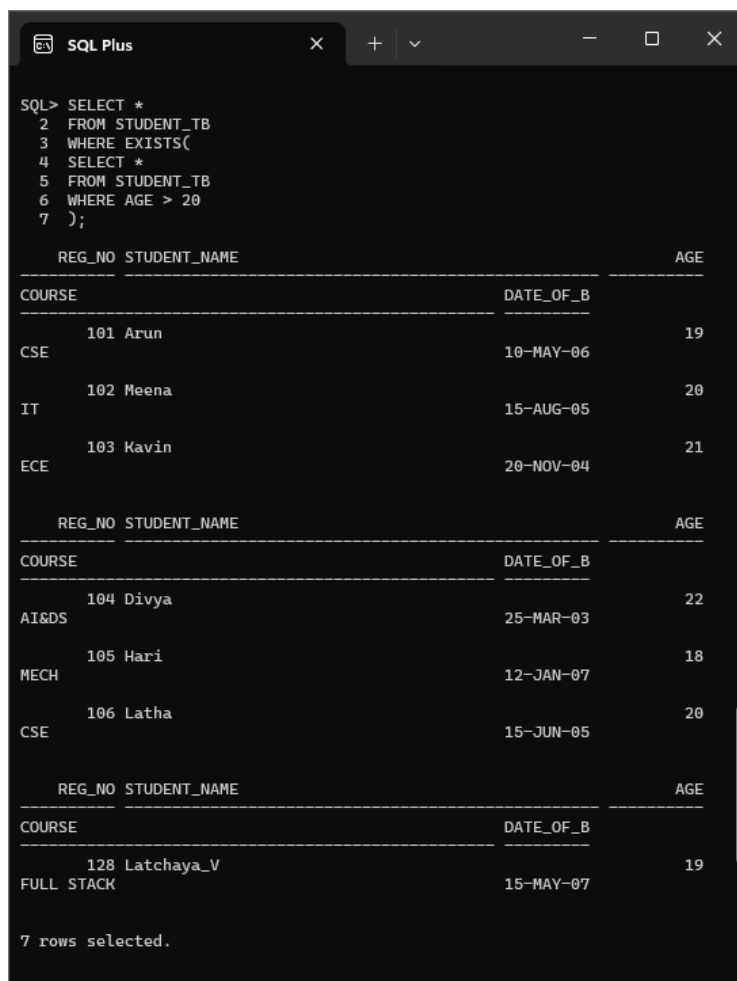
- If subquery returns at least one row → TRUE → ALL ROWS WILL DISPLAY
- If no rows → FALSE → NO ROWS WILL DISPLAY

Syntax

```
SELECT *  
FROM table1  
WHERE EXISTS(subquery);
```

Example

```
SELECT *  
FROM STUDENT_TB  
WHERE EXISTS(  
SELECT *  
FROM STUDENT_TB  
WHERE AGE > 20  
);
```



The screenshot shows a SQL Plus window with a query that uses the EXISTS operator to filter students whose age is greater than 20. The query is as follows:

```
SQL> SELECT *  
2 FROM STUDENT_TB  
3 WHERE EXISTS(  
4 SELECT *  
5 FROM STUDENT_TB  
6 WHERE AGE > 20  
7 );
```

The result displays a list of students and their details, grouped into three sections. Each section has a header with columns: REG_NO, STUDENT_NAME, COURSE, DATE_OF_B, and AGE. The first section contains three rows (Arun, Meena, and Kavin), the second section contains three rows (Divya, Hari, and Latha), and the third section contains one row (Latchaya_V). The total number of rows selected is 7.

REG_NO	STUDENT_NAME	COURSE	DATE_OF_B	AGE
101	Arun	CSE	10-MAY-06	19
102	Meena	IT	15-AUG-05	20
103	Kavin	ECE	20-NOV-04	21
104	Divya	AI&DS	25-MAR-03	22
105	Hari	MECH	12-JAN-07	18
106	Latha	CSE	15-JUN-05	20
128	Latchaya_V	FULL STACK	15-MAY-07	19

7 rows selected.

15. ANY

Definition

Compares a value with any value returned by subquery.

ANY checks whether the condition is true for at least one value returned by the subquery.

- If condition matches at least one value → TRUE
- If condition matches no values → FALSE

Syntax

```
SELECT *  
FROM table_name  
WHERE column_name operator ANY(subquery);
```

Example

```
SELECT *  
FROM STUDENT_TB  
WHERE AGE > ANY(  
SELECT AGE  
FROM STUDENT_TB  
WHERE COURSE = 'CSE'  
);
```

Important Shortcut

Operator	Equivalent
> ANY(values)	Greater than minimum value
< ANY(values)	Less than maximum value

Here:

minimum = 19

So: AGE > ANY(19,20)

effectively behaves like:

AGE > 19

```
SQL Plus
SQL> SELECT *
2 FROM STUDENT_TB
3 WHERE AGE > ANY(
4 SELECT AGE
5 FROM STUDENT_TB
6 WHERE COURSE = 'CSE'
7 );
```

	REG_NO	STUDENT_NAME		AGE
COURSE			DATE_OF_B	
IT	102	Meena	15-AUG-05	20
ECE	103	Kavin	20-NOV-04	21
AI&DS	104	Divya	25-MAR-03	22
CSE	106	Latha	15-JUN-05	20

```
SQL> |
```

16. ALL

Definition

ONLY RETURNS THE ROWS THAT SATIFY ALL CONDITIONS

Compares a value with all values returned by subquery.

ALL checks whether the condition is true for every value returned by the subquery.

- If condition matches all values → TRUE
- If condition fails for even one value → FALSE

Syntax

```
SELECT *
```

```
FROM table_name
```

```
WHERE column_name operator ALL(subquery);
```

Example

```
SELECT *
```

```
FROM STUDENT_TB
```

```
WHERE AGE > ALL(
```

```
SELECT AGE
```

```
FROM STUDENT_TB
```

```
WHERE COURSE = 'CSE'
```

```
);
```

```
SQL Plus
SQL> SELECT *
  2 FROM STUDENT_TB
  3 WHERE AGE > ALL(
  4 SELECT AGE
  5 FROM STUDENT_TB
  6 WHERE COURSE = 'CSE'
  7 );
```

REG_NO	STUDENT_NAME	AGE
103	Kavin	21
104	Divya	22

```
SQL>
```

17. UNION

Definition

Combines results of two queries and removes duplicates.

Syntax

```
SELECT column_name
FROM table1
UNION
SELECT column_name
FROM table2;
```

Example

```
SELECT COURSE
FROM STUDENT_TB
UNION
SELECT COURSE
FROM STUDENT_BACKUP;
```

```
SQL Plus
1 row created.

SQL> SELECT COURSE
  2 FROM STUDENT_TB
  3 UNION
  4 SELECT COURSE
  5 FROM STUDENT_BACKUP;
```

COURSE
CSE
IT
ECE
AI&DS
MECH
FULL STACK
CYBER SECURITY

```
SQL>
```

18. INTERSECT

Definition

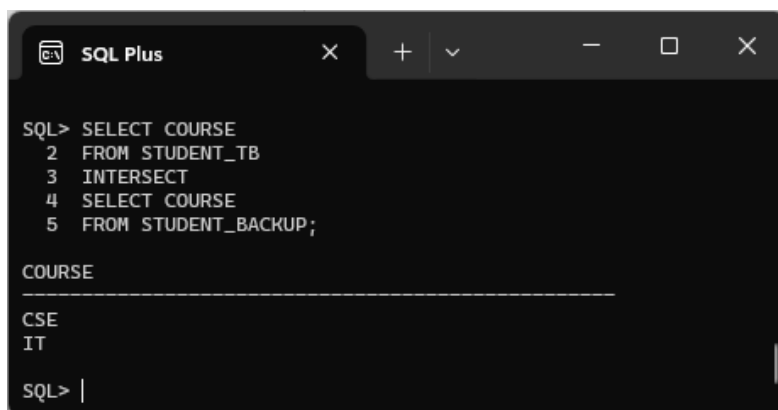
Returns common rows from two queries.

Syntax

```
SELECT column_name  
FROM table1  
INTERSECT  
SELECT column_name  
FROM table2;
```

Example

```
SELECT COURSE  
FROM STUDENT_TB  
INTERSECT  
SELECT COURSE  
FROM STUDENT_BACKUP;
```



The screenshot shows a terminal window titled "SQL Plus". The prompt "SQL>" is followed by the query: "SELECT COURSE", "2 FROM STUDENT_TB", "3 INTERSECT", "4 SELECT COURSE", "5 FROM STUDENT_BACKUP;". Below the query, the results are displayed under the heading "COURSE". The results are "CSE" and "IT". The prompt "SQL>" is followed by a vertical bar "|".

19. MINUS / EXCEPT

Definition

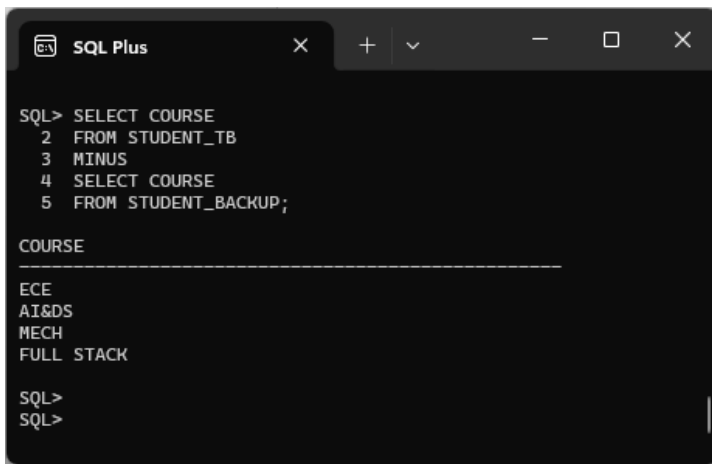
Returns rows present in first query but not in second query.
It also doesn't display the common rows

Oracle Syntax (MINUS)

```
SELECT column_name  
FROM table1  
MINUS  
SELECT column_name  
FROM table2;
```

Oracle Example

```
SELECT COURSE  
FROM STUDENT_TB  
MINUS  
SELECT COURSE  
FROM STUDENT_BACKUP;
```



```
SQL> SELECT COURSE
2 FROM STUDENT_TB
3 MINUS
4 SELECT COURSE
5 FROM STUDENT_BACKUP;

COURSE
-----
ECE
AI&DS
MECH
FULL STACK

SQL>
SQL>
```

MySQL Syntax (EXCEPT not supported in old versions)

```
SELECT column_name
FROM table1
EXCEPT
SELECT column_name
FROM table2;
```

20. AND

Definition

Returns records only if all conditions are true.

Syntax

```
SELECT *
FROM table_name
WHERE condition1 AND condition2;
```

Example

```
SELECT *
FROM STUDENT_TB
WHERE AGE > 18
AND COURSE = 'IT';
```



```
SQL> SELECT *
2 FROM STUDENT_TB
3 WHERE AGE > 18
4 AND COURSE = 'IT';

REG_NO STUDENT_NAME                                AGE
-----
COURSE                                DATE_OF_B
-----
IT      102 Meena                                20
        15-AUG-05
```

21. OR

Definition

Returns records if at least one condition is true.

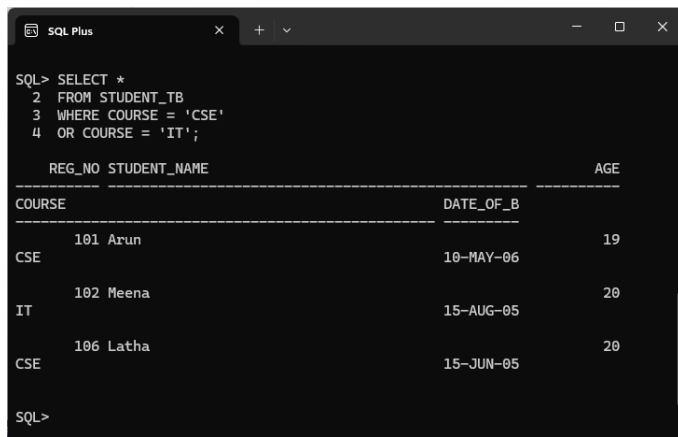
Syntax

```
SELECT *
```

FROM table_name
WHERE condition1 OR condition2;

Example

```
SELECT *  
FROM STUDENT_TB  
WHERE COURSE = 'CSE'  
OR COURSE = 'IT';
```



The screenshot shows a terminal window titled 'SQL Plus' with the following SQL query entered:

```
SQL> SELECT *  
2 FROM STUDENT_TB  
3 WHERE COURSE = 'CSE'  
4 OR COURSE = 'IT';
```

The output is a table with the following data:

	REG_NO	STUDENT_NAME	AGE
COURSE			
	101	Arun	19
CSE			
	102	Meena	20
IT			
	106	Latha	20
CSE			

The terminal prompt 'SQL>' is visible at the bottom.

23. NOT

Definition

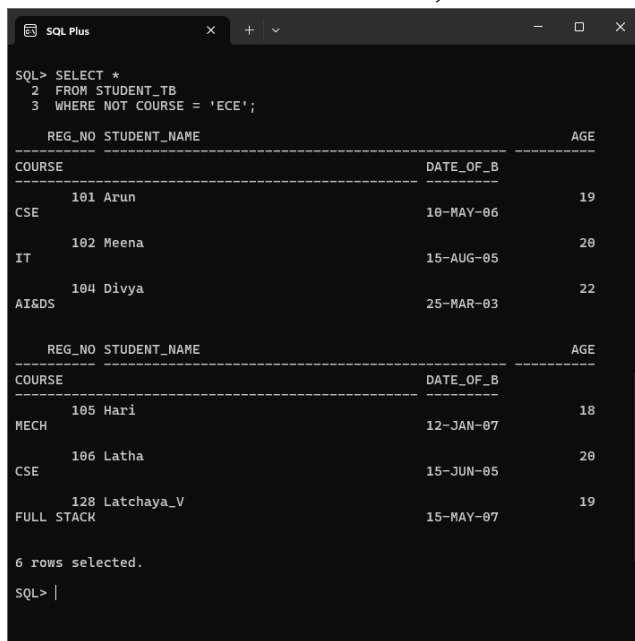
Negates a condition.

Syntax

```
SELECT *  
FROM table_name  
WHERE NOT condition;
```

Example

```
SELECT *  
FROM STUDENT_TB  
WHERE NOT COURSE = 'ECE';
```



The screenshot shows a terminal window titled 'SQL Plus' with the following SQL query entered:

```
SQL> SELECT *  
2 FROM STUDENT_TB  
3 WHERE NOT COURSE = 'ECE';
```

The output is a table with the following data:

	REG_NO	STUDENT_NAME	AGE
COURSE			
	101	Arun	19
CSE			
	102	Meena	20
IT			
	104	Divya	22
AI&DS			

	REG_NO	STUDENT_NAME	AGE
COURSE			
	105	Hari	18
MECH			
	106	Latha	20
CSE			
	128	Latchaya_V	19
FULL STACK			

The terminal shows '6 rows selected.' and the prompt 'SQL> |' at the bottom.

Operator	Short Description
>	Checks values greater than another value
<	Checks values less than another value
>=	Checks values greater than or equal to another value
<=	Checks values less than or equal to another value
<>	Checks values that are not equal
=	Checks values that are equal
BETWEEN	Checks values within a range
NOT BETWEEN	Checks values outside a range
IN	Checks whether value exists in a list
NOT IN	Checks whether value does not exist in a list
IS NULL	Checks NULL/empty values
IS NOT NULL	Checks non-null values
LIKE	Used for pattern matching
EXISTS	Checks whether subquery returns rows
ANY	Condition must match at least one subquery value
ALL	Condition must match all subquery values
UNION	Combines results and removes duplicates
INTERSECT	Returns common rows from queries
MINUS / EXCEPT	Returns rows from first query excluding common rows
AND	All conditions must be true
OR	At least one condition must be true
NOT	Negates/reverses a condition

REFERENTIAL INTEGRITY

(Referential Integrity ensures that a foreign key value in one table must exist as a primary key value in another related table.)

Definition

Referential Integrity is a rule that maintains consistency between related tables using a **FOREIGN KEY**.

☞ It ensures that a foreign key value in one table must exist as a primary key in another table.

☞ Prevents invalid or orphan records → (child table records that reference a parent record which does not exist)

Main Idea

- Parent table → contains PRIMARY KEY
- Child table → contains FOREIGN KEY

Foreign key values must match existing primary key values.

Purpose of Referential Integrity

- ✓ Prevents invalid data
- ✓ Maintains relationship consistency
- ✓ Avoids orphan records
- ✓ Ensures data accuracy between tables

Real-Life Example

Table	Purpose
DEPARTMENT	Parent table
STUDENT	Child table

A student can belong only to an existing department.

Example

Parent Table

```
CREATE TABLE DEPARTMENT(  
  DEPT_ID INT PRIMARY KEY,  
  DEPT_NAME VARCHAR2(50)  
);
```

Child Table

```
CREATE TABLE STUDENT(  
  REG_NO INT,  
  STUDENT_NAME VARCHAR2(50),  
  DEPT_ID INT,  
  CONSTRAINT fk  
  FOREIGN KEY(DEPT_ID)  
  REFERENCES DEPARTMENT(DEPT_ID)  
);
```

Meaning

FOREIGN KEY(DEPT_ID)

REFERENCES DEPARTMENT(DEPT_ID)

☞ DEPT_ID in STUDENT table must exist in DEPARTMENT table.

Valid Insert

INSERT INTO DEPARTMENT

VALUES (101, 'CSE');

INSERT INTO STUDENT

VALUES (1, 'Arun', 101);

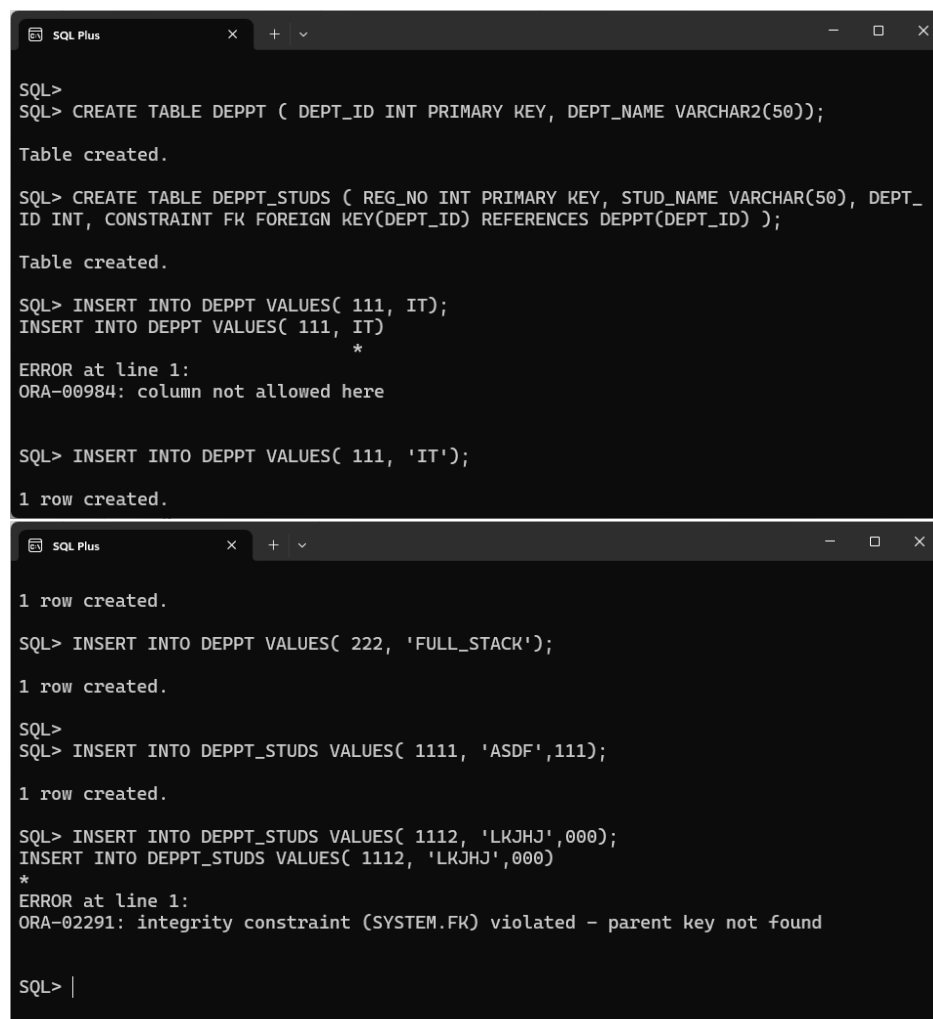
✓ Works because department 101 exists.

Invalid Insert

INSERT INTO STUDENT

VALUES (2, 'Meena', 105);

✗ Error because department 105 does not exist.



```
SQL>
SQL> CREATE TABLE DEPPT ( DEPT_ID INT PRIMARY KEY, DEPT_NAME VARCHAR2(50));

Table created.

SQL> CREATE TABLE DEPPT_STUDS ( REG_NO INT PRIMARY KEY, STUD_NAME VARCHAR(50), DEPT_
ID INT, CONSTRAINT FK FOREIGN KEY(DEPT_ID) REFERENCES DEPPT(DEPT_ID) );

Table created.

SQL> INSERT INTO DEPPT VALUES( 111, IT);
INSERT INTO DEPPT VALUES( 111, IT)
      *
ERROR at line 1:
ORA-00984: column not allowed here

SQL> INSERT INTO DEPPT VALUES( 111, 'IT');

1 row created.

1 row created.

SQL> INSERT INTO DEPPT VALUES( 222, 'FULL_STACK');

1 row created.

SQL>
SQL> INSERT INTO DEPPT_STUDS VALUES( 1111, 'ASDF',111);

1 row created.

SQL> INSERT INTO DEPPT_STUDS VALUES( 1112, 'LKJHJ',000);
INSERT INTO DEPPT_STUDS VALUES( 1112, 'LKJHJ',000)
      *
ERROR at line 1:
ORA-02291: integrity constraint (SYSTEM.FK) violated - parent key not found

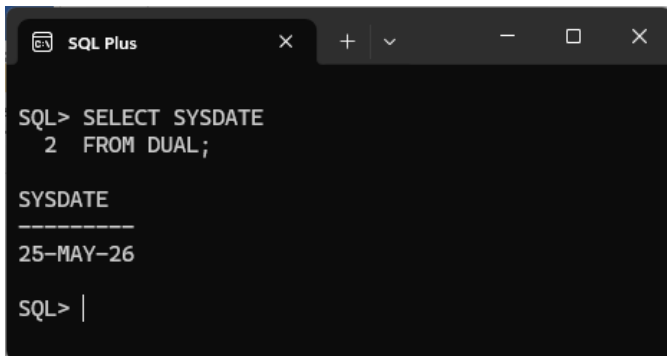
SQL>
```

DATE & TIME

Part	Purpose
DUAL	Special dummy table in Oracle

1. Display Current

```
SELECT SYSDATE  
FROM DUAL;
```



```
SQL> SELECT SYSDATE  
2 FROM DUAL;  
  
SYSDATE  
-----  
25-MAY-26  
  
SQL> |
```

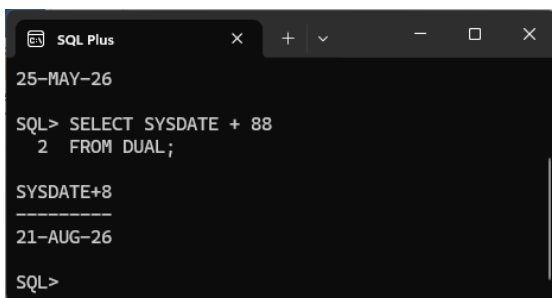
Insert Current Date Automatically

```
INSERT INTO STUDENT_TB  
VALUES (107, 'Ravi', 21, 'ECE', SYSDATE);
```

2. Add Days to Current Date

```
SELECT SYSDATE + 88  
FROM DUAL;
```

☞ Displays date after 88 days.

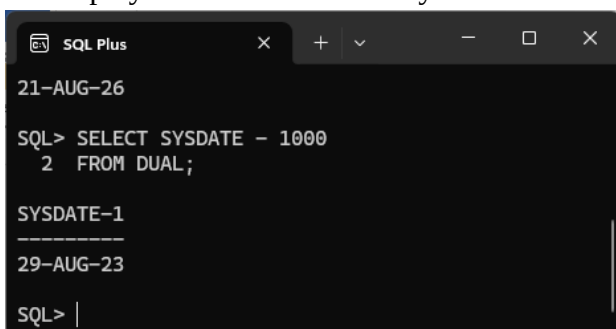


```
25-MAY-26  
  
SQL> SELECT SYSDATE + 88  
2 FROM DUAL;  
  
SYSDATE+88  
-----  
21-AUG-26  
  
SQL> |
```

3. Subtract Days

```
SELECT SYSDATE - 1000  
FROM DUAL;
```

☞ Displays date before 1000 days.

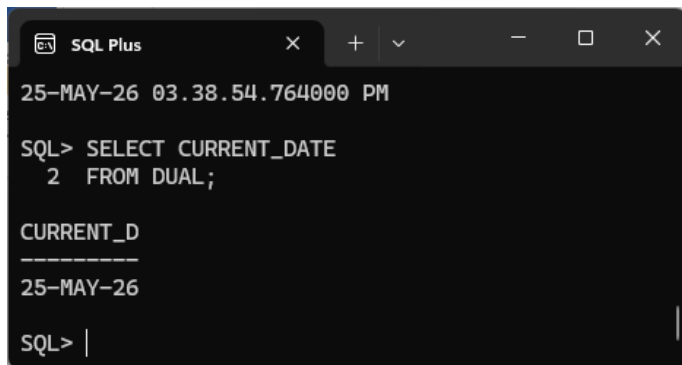


```
21-AUG-26  
  
SQL> SELECT SYSDATE - 1000  
2 FROM DUAL;  
  
SYSDATE-1  
-----  
29-AUG-23  
  
SQL> |
```

4. CURRENT_DATE

☞ Returns current date in session time zone.

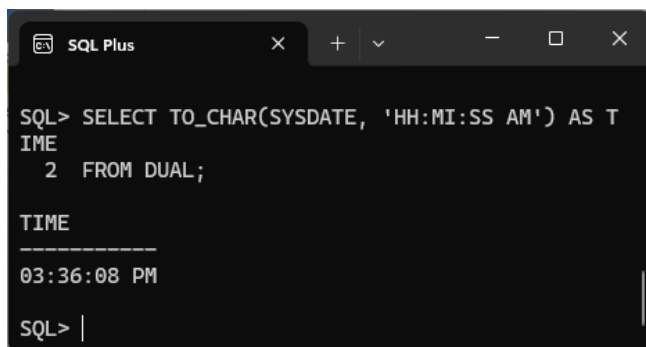
```
SELECT CURRENT_DATE  
FROM DUAL;
```



```
SQL Plus  
25-MAY-26 03.38.54.764000 PM  
  
SQL> SELECT CURRENT_DATE  
2 FROM DUAL;  
  
CURRENT_D  
-----  
25-MAY-26  
  
SQL> |
```

5. Current Time Only

```
SELECT TO_CHAR(SYSDATE, 'HH:MI:SS AM') AS TIME  
FROM DUAL;
```

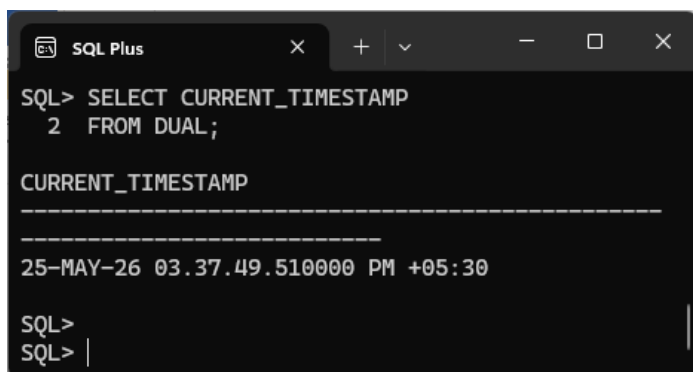


```
SQL Plus  
  
SQL> SELECT TO_CHAR(SYSDATE, 'HH:MI:SS AM') AS T  
IME  
2 FROM DUAL;  
  
TIME  
-----  
03:36:08 PM  
  
SQL> |
```

6. CURRENT_TIMESTAMP

☞ Returns current date, time, and time zone.

```
SELECT CURRENT_TIMESTAMP  
FROM DUAL;
```

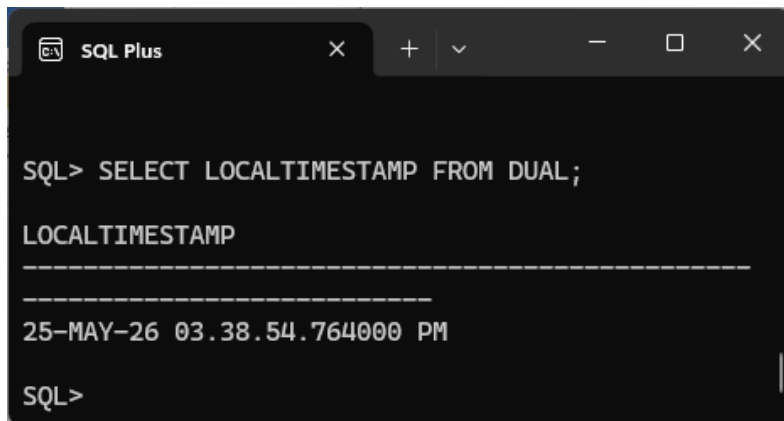


```
SQL Plus  
  
SQL> SELECT CURRENT_TIMESTAMP  
2 FROM DUAL;  
  
CURRENT_TIMESTAMP  
-----  
-----  
25-MAY-26 03.37.49.510000 PM +05:30  
  
SQL>  
SQL> |
```

7. LOCALTIMESTAMP

☞ Returns local timestamp without time zone.

```
SELECT LOCALTIMESTAMP  
FROM DUAL;
```

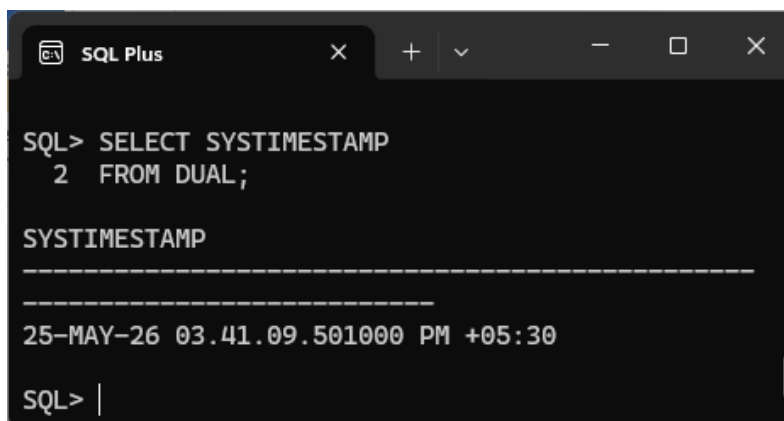


```
SQL> SELECT LOCALTIMESTAMP FROM DUAL;  
  
LOCALTIMESTAMP  
-----  
25-MAY-26 03.38.54.764000 PM  
  
SQL>
```

8. SYSTIMESTAMP

☞ Returns system date, time, fractional seconds, and time zone.

```
SELECT SYSTIMESTAMP  
FROM DUAL;
```



```
SQL> SELECT SYSTIMESTAMP  
2 FROM DUAL;  
  
SYSTIMESTAMP  
-----  
25-MAY-26 03.41.09.501000 PM +05:30  
  
SQL> |
```

MONTH & DATE FUNCTIONS (Oracle SQL)

1. MONTHS_BETWEEN()

Definition

Returns the number of months between two dates.

Syntax

```
SELECT MONTHS_BETWEEN(date1, date2)  
FROM DUAL;
```

Example

```
SELECT MONTHS_BETWEEN('26-MAY-26','16-JAN-25')  
FROM DUAL;
```

```
SQL Plus
SQL> SELECT MONTHS_BETWEEN('26-MAY-26', '16-
2  JAN-25')
      FROM DUAL;

MONTHS_BETWEEN('26-MAY-26', '16-JAN-25')
-----
16.3225806

SQL> |
```

2. ADD_MONTHS()

Definition

Adds a specified number of months to a given date.

Syntax

```
SELECT ADD_MONTHS(date, number_of_months)
FROM DUAL;
```

Example

```
SELECT ADD_MONTHS('16-JAN-25', 16)
FROM DUAL;
```

```
SQL Plus
16.3225806

SQL> SELECT ADD_MONTHS('16-JAN-25', 16)
2  FROM DUAL;

ADD_MONTH
-----
16-MAY-26

SQL>
```

3. NEXT_DAY()

Definition

Returns the next occurrence of a specified weekday after a given date.

Syntax

```
SELECT NEXT_DAY(date, 'day_name')
FROM DUAL;
```

Example

```
SELECT NEXT_DAY('26-MAY-26', 'MONDAY')
FROM DUAL;
```

```
SQL Plus
16-MAY-26

SQL> SELECT NEXT_DAY('26-MAY-26', 'MONDAY')
2 FROM DUAL;

NEXT_DAY(
-----
01-JUN-26

SQL>
```

4. LAST_DAY()

Definition

Returns the last date of the month for a given date.

Syntax

```
SELECT LAST_DAY(date)
FROM DUAL;
```

Example

```
SELECT LAST_DAY('26-MAY-26')
FROM DUAL;
```

```
SQL Plus
01-JUN-26

SQL> SELECT LAST_DAY('26-MAY-26')
2 FROM DUAL;

LAST_DAY(
-----
31-MAY-26

SQL> |
```

5. DATE SUBTRACTION (DAYS BETWEEN DATES)

Definition

In Oracle, subtracting one date from another returns the **number of days between the two dates**.

☞ There is no built-in DAYS_BETWEEN() function.

☞ Instead, we directly use DATE - DATE.

Syntax

```
SELECT date1 - date2
FROM DUAL;
```

Example

```
SELECT TO_DATE('31-MAY-26','DD-MON-YY')
      - TO_DATE('16-JAN-25','DD-MON-YY') AS DAYS
FROM DUAL;
```

```
SQL Plus
SQL> SELECT TO_DATE('31-MAY-26','DD-MON-YY')
2      - TO_DATE('16-JAN-25','DD-MON-YY') AS DAYS
3 FROM DUAL;

      DAYS
-----
      500

SQL> |
```

Function	Purpose
MONTHS_BETWEEN	Finds difference between two dates in months
ADD_MONTHS	Adds months to a date
NEXT_DAY	Finds next weekday date
LAST_DAY	Finds last date of month

CHARACTER FUNCTIONS

☞ Character functions are used to **manipulate and format string data in SQL tables**

1. UPPER()

Definition

Converts all characters of a string to **uppercase**.

Syntax

```
SELECT UPPER(column_name)
FROM table_name;
```

Example

```
SELECT UPPER(STUDENT_NAME) AS NAME_IN_UPPER_CASE
FROM STUDENT_TB;
```

```
SQL Plus
SQL> SELECT UPPER(STUDENT_NAME) AS NAME_IN_UPPER_CA
SE
2 FROM STUDENT_TB;

NAME_IN_UPPER_CASE
-----
ARUN
MEENA
KAVIN
DIVYA
HARI
LATHA
LATCHAYA_V

7 rows selected.

SQL>
```

2. LOWER()

Definition

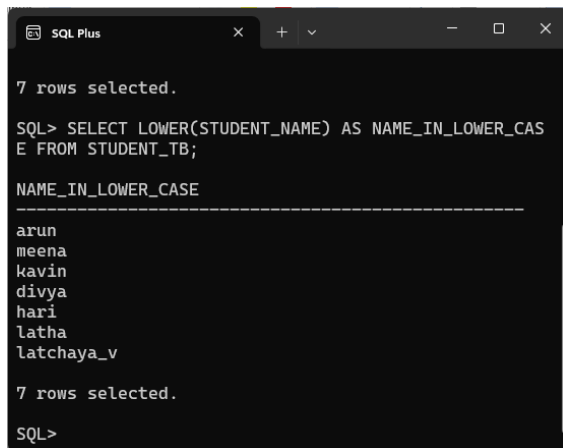
Converts all characters of a string to **lowercase**.

Syntax

```
SELECT LOWER(column_name)
FROM table_name;
```

Example

```
SELECT LOWER(STUDENT_NAME) AS NAME_IN_LOWER_CASE
FROM STUDENT_TB;
```



```
SQL Plus
7 rows selected.
SQL> SELECT LOWER(STUDENT_NAME) AS NAME_IN_LOWER_CASE
FROM STUDENT_TB;
NAME_IN_LOWER_CASE
-----
arun
meena
kavin
divya
hari
latha
latchaya_v
7 rows selected.
SQL>
```

3. INITCAP()

Definition

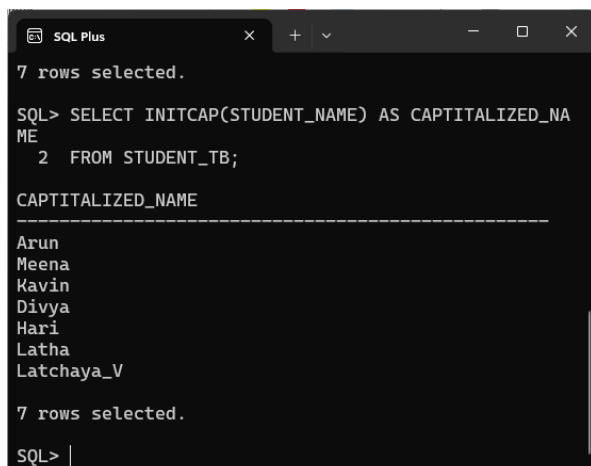
Capitalizes the **first letter of each word**.

Syntax

```
SELECT INITCAP(column_name)
FROM table_name;
```

Example

```
SELECT INITCAP(STUDENT_NAME) AS CAPTITALIZED_NAME
FROM STUDENT_TB;
```



```
SQL Plus
7 rows selected.
SQL> SELECT INITCAP(STUDENT_NAME) AS CAPTITALIZED_NAME
FROM STUDENT_TB;
CAPTITALIZED_NAME
-----
Arun
Meena
Kavin
Divya
Hari
Latha
Latchaya_V
7 rows selected.
SQL> |
```

4. LENGTH()

Definition

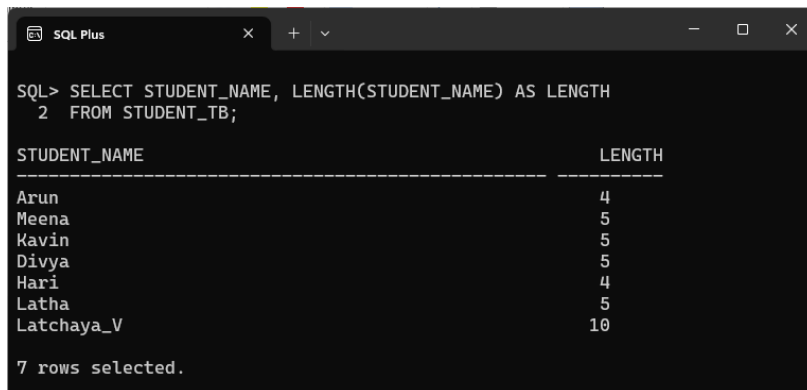
Returns number of characters in a string.

Syntax

```
SELECT LENGTH(column_name)
FROM table_name;
```

Example

```
SELECT STUDENT_NAME, LENGTH(STUDENT_NAME) AS LENGTH
FROM STUDENT_TB;
```



```
SQL Plus
SQL> SELECT STUDENT_NAME, LENGTH(STUDENT_NAME) AS LENGTH
2  FROM STUDENT_TB;

STUDENT_NAME          LENGTH
-----
Arun                   4
Meena                  5
Kavin                  5
Divya                  5
Hari                   4
Latha                  5
Latchaya_V            10

7 rows selected.
```

5. SUBSTR()

Definition

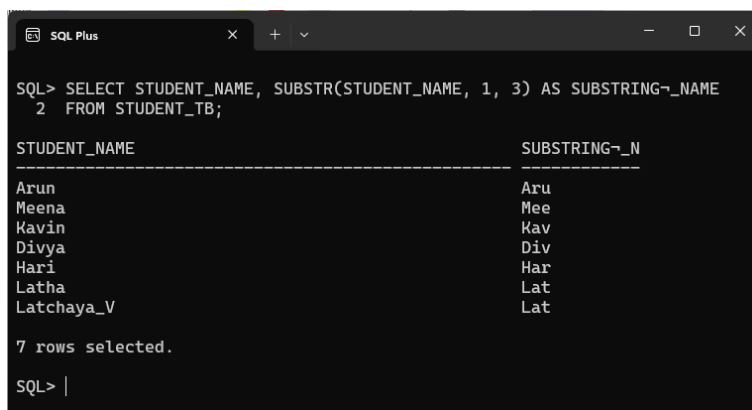
Extracts part of a string.

Syntax

```
SELECT SUBSTR(column_name, start_position, length)
FROM table_name;
```

Example

```
SELECT STUDENT_NAME, SUBSTR(STUDENT_NAME, 1, 3) AS SUBSTRING_NAME
FROM STUDENT_TB;
```



```
SQL Plus
SQL> SELECT STUDENT_NAME, SUBSTR(STUDENT_NAME, 1, 3) AS SUBSTRING_NAME
2  FROM STUDENT_TB;

STUDENT_NAME          SUBSTRING_NAME
-----
Arun                   Aru
Meena                  Mee
Kavin                  Kav
Divya                  Div
Hari                   Har
Latha                  Lat
Latchaya_V            Lat

7 rows selected.

SQL> |
```

6. INSTR()

Definition

Returns position of a character in a string.

Case Sensitive .

Syntax

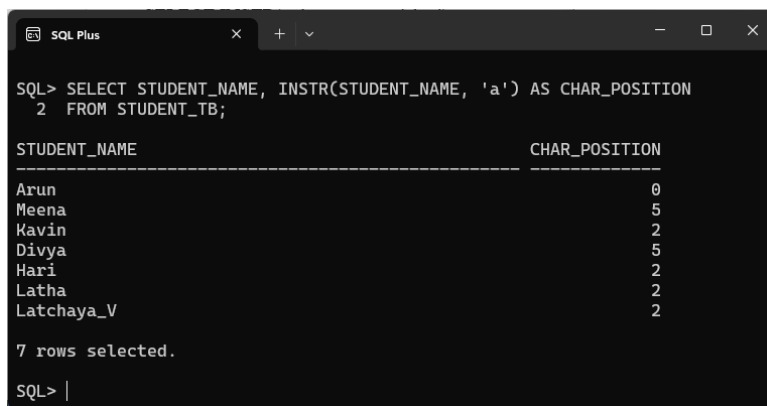
```
SELECT INSTR(column_name, 'char')
```

```
FROM table_name;
```

Example

```
SELECT STUDENT_NAME, INSTR(STUDENT_NAME, 'a') AS CHAR_POSITION
```

```
FROM STUDENT_TB;
```



The screenshot shows a SQL Plus window with the following content:

```
SQL> SELECT STUDENT_NAME, INSTR(STUDENT_NAME, 'a') AS CHAR_POSITION
2  FROM STUDENT_TB;
```

STUDENT_NAME	CHAR_POSITION
Arun	0
Meena	5
Kavin	2
Divya	5
Hari	2
Latha	2
Latchaya_V	2

7 rows selected.

```
SQL> |
```

7. REPLACE()

Definition

Replaces a character or word in a string.

Case Sensitive .

Syntax

```
SELECT REPLACE(column_name, 'old', 'new')
```

```
FROM table_name;
```

Example

```
SELECT STUDENT_NAME, REPLACE(STUDENT_NAME, 'a', '@') AS REPLACED_NAME
```

```
FROM STUDENT_TB;
```

```
SQL Plus
SQL> SELECT STUDENT_NAME, REPLACE(STUDENT_NAME, 'a', '@') AS REPLACED_NAME
2 FROM STUDENT_TB;

STUDENT_NAME
-----
REPLACED_NAME
-----
Arun
Arun
Meena
Meen@
Kavin
K@vin

STUDENT_NAME
-----
REPLACED_NAME
-----
Divya
Divy@
Hari
H@ri
Latha
L@th@

STUDENT_NAME
-----
REPLACED_NAME
-----
Latchaya_V
L@tch@y@_V

7 rows selected.

SQL> |
```

8. CONCAT()

Definition

Joins two strings together.

Syntax

```
SELECT CONCAT(col1, col2)
FROM table_name;
```

Example

```
SELECT STUDENT_NAME, COURSE,
       CONCAT(STUDENT_NAME, COURSE) AS CONCATENATED
FROM STUDENT_TB;
```

```
SQL Plus
SQL> SELECT STUDENT_NAME, COURSE,
2         CONCAT(STUDENT_NAME, COURSE) AS CONCATENATED
3 FROM STUDENT_TB;

STUDENT_NAME
-----
COURSE
-----
CONCATENATED
-----
Arun
CSE
ArunCSE

Meena
IT
MeenaIT

STUDENT_NAME
-----
COURSE
-----
CONCATENATED
-----
Kavin
ECE
KavinECE

Divya
AI&DS
```

```

STUDENT_NAME
-----
COURSE
-----
CONCATENATED
-----
DivyaAI&DS

Hari
MECH
HarIMECH

Latha

STUDENT_NAME
-----
COURSE
-----
CONCATENATED
-----
CSE
LathaCSE

Latchaya_V
FULL STACK
Latchaya_VFULL STACK

7 rows selected.

SQL> |

```

9. TRIM()

Definition

Removes spaces from both sides of a string.

Syntax

```

SELECT TRIM(' string ')
FROM DUAL;

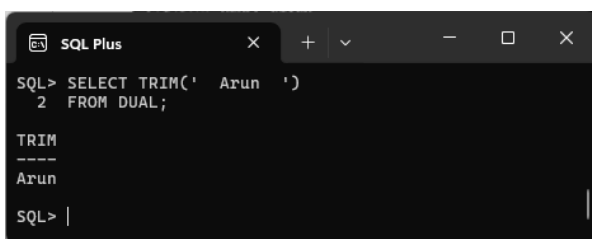
```

Example

```

SELECT TRIM(' Arun ')
FROM DUAL;

```



```

SQL Plus
SQL> SELECT TRIM(' Arun ')
2 FROM DUAL;

TRIM
----
Arun

SQL> |

```

10. LTRIM() / RTRIM()

Definition

Removes spaces from left / right side.

Syntax

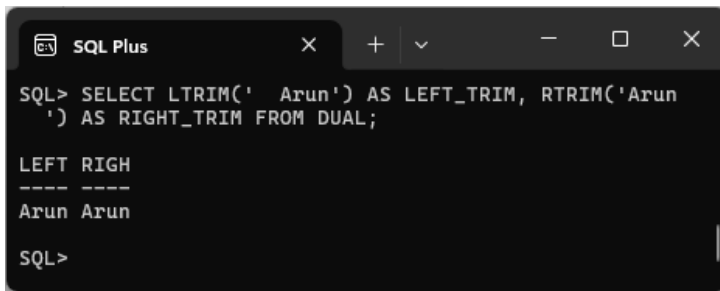
```

SELECT LTRIM(' text ')
FROM DUAL;
SELECT RTRIM(' text ')
FROM DUAL;

```

Example

```
SELECT LTRIM(' Arun') FROM DUAL;  
SELECT RTRIM('Arun ') FROM DUAL;
```



```
SQL Plus  
SQL> SELECT LTRIM(' Arun') AS LEFT_TRIM, RTRIM('Arun  
' ) AS RIGHT_TRIM FROM DUAL;  
  
LEFT TRIM  RIGHT TRIM  
-----  
Arun      Arun  
  
SQL>
```

11. LPAD()

Definition

LPAD adds characters to the **left side** of a string until it reaches a specified length.

Syntax

```
SELECT LPAD(column_name, total_length, 'pad_string')  
FROM table_name;
```

Example (using STUDENT_TB)

```
SELECT STUDENT_NAME,  
       LPAD(STUDENT_NAME, 15, '*') AS LEFT_PADDED  
FROM STUDENT_TB;
```



```
SQL Plus  
SQL> SELECT STUDENT_NAME,  
2         LPAD(STUDENT_NAME, 15, '*') AS LEFT_PADDED  
3         FROM STUDENT_TB;  
  
STUDENT_NAME  
-----  
LEFT_PADDED  
-----  
Arun  
*****Arun  
  
Meena  
*****Meena  
  
Kavin  
*****Kavin  
  
STUDENT_NAME  
-----  
LEFT_PADDED  
-----  
Divya  
*****Divya  
  
Hari  
*****Hari  
  
Latha  
*****Latha  
  
STUDENT_NAME  
-----  
LEFT_PADDED  
-----  
Latchaya_V  
*****Latchaya_V  
  
7 rows selected.  
  
SQL>
```

12. RPAD()

Definition

RPAD adds characters to the **right side** of a string until it reaches a specified length.

Syntax

```
SELECT RPAD(column_name, total_length, 'pad_string')
FROM table_name;
```

Example

```
SELECT STUDENT_NAME,
       RPAD(STUDENT_NAME, 15, '*') AS RIGHT_PADDED
FROM STUDENT_TB;
```



The screenshot shows a SQL Plus window with the following output:

```
7 rows selected.
SQL> SELECT STUDENT_NAME,
2          RPAD(STUDENT_NAME, 15, '*') AS RIGHT_PADDED
3          FROM STUDENT_TB;

STUDENT_NAME
-----
RIGHT_PADDED
-----
Arun
Arun*****

Meena
Meena*****

Kavin
Kavin*****

STUDENT_NAME
-----
RIGHT_PADDED
-----
Divya
Divya*****

Hari
Hari*****

Latha
Latha*****

STUDENT_NAME
-----
RIGHT_PADDED
-----
Latchaya_V
Latchaya_V*****

7 rows selected.
SQL> |
```